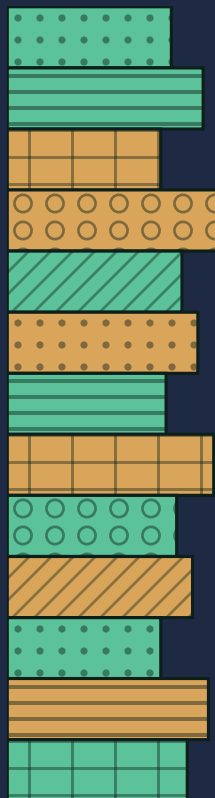


GENERAL EDITION · A FIELD GUIDE FOR ANYONE
WHO CAN USE A COMPUTER BUT WANTS TO
UNDERSTAND ONE

The Missing Context

How computers actually work

*Everyday examples: photos, bank statements, group
chats and the weather. Fifteen chapters, each one a
step deeper than the first edition.*



Shivam Meena

The Missing Context

How computers actually work

The Missing Context: How Computers Actually Work

General edition

Shivam Meena

General edition, September 2026. Based on the first edition, written for Neha.

Companion site, with an assessment, a playground and a lab for every chapter:

<https://strata.theether.in>



Links in this PDF are clickable. Figures were drawn for this book. The expenses in Chapter 15 are invented for practice. Set in Lora, Bitstream Charter and DejaVu Sans Mono.

For everyone who was never handed the manual.

Contents

	Preface: Why this book exists	6
CHAPTER 1	What a computer actually is	8
CHAPTER 2	The operating system	12
CHAPTER 3	Files, folders and paths	16
CHAPTER 4	The terminal	21
CHAPTER 5	Setting up your computer	25
CHAPTER 6	What a programming language actually is	30
CHAPTER 7	Libraries, packages and package managers	35
CHAPTER 8	Versions and environments	39
CHAPTER 9	Networks, the internet and APIs	44
CHAPTER 10	Data: spreadsheets, CSV, JSON and databases	49
CHAPTER 11	Errors, and how to find answers	54
CHAPTER 12	Staying safe	59
CHAPTER 13	Learning without getting lost	64
CHAPTER 14	Saving your history with Git	68
CHAPTER 15	Capstone: a month of expenses, end to end	72
	Afterword: your own project	78
	Appendix: What you should see	80
	Cheat sheet: the commands you will keep looking up	83
	Glossary	86
	Further reading and books to grow into	90

Preface: Why this book exists

You already use a computer all day. You send messages, edit photos, fill in forms, download things, and occasionally shout at it. What you may never have been given is the *context*: what a file actually is, what the terminal is for, why a program you downloaded works on one laptop and not another, and what "install" and "the cloud" really mean. Most courses that teach programming skip all of that. They start with how to write a loop and assume you already know where the loop lives.

This book fills in that background. It does not teach you a programming language. It teaches you the world that programs live in, so that when you learn a language, or read an error, or follow a tutorial, the pieces have somewhere to attach.

Who this is for

Anyone who can use a computer but wants to understand one: students starting a computing course, professionals who keep hitting the same walls, people who want to automate a small chore, and anyone who has ever felt that everyone else was handed a manual they didn't get. No mathematics beyond arithmetic is needed.

How to read this book

- **Read it in order.** Each chapter builds on the one before it. Later chapters make sense only once the earlier ones are in place.

- **Do the "Try it" boxes.** Reading about the terminal teaches you far less than typing three commands into one. Each exercise takes a few minutes and uses things already on your computer: photos, downloads, messages.
- **Read "Going deeper" on the second pass.** Every chapter ends with a section that goes one level further down. Skip it the first time through if it feels like too much; come back when the rest is comfortable.
- **Don't memorise.** Aim to understand what each thing is *for*. Names and commands come with repetition, and Chapter 11 shows you how to look anything up.
- **Expect to re-read.** Some chapters will mean much more after you have written a little code. That is normal, not a sign you missed something.

The examples

Everything here is illustrated with ordinary life: a kitchen for the computer, a building manager for the operating system, postal addresses for file paths, food delivery for the internet, a month of expenses for the final project. Analogies are scaffolding, not truth. When one stops fitting, drop it and keep the idea underneath.

Instructions are written for Windows first, with notes for Mac and Linux where they differ. Where a Windows path such as `C:\Users\you\Documents` appears, `you` stands for your own user name.

About this edition

The first edition of this book was written for one reader, a geologist named Neha, and its examples were rocks and boreholes. This edition keeps the same structure and adds two chapters, on data and on staying safe, plus a deeper section at the end of every chapter. A companion website with a five-question assessment, a browser playground and a hands-on lab for each chapter lives at strata.theether.in; its examples still lean towards the first edition, and a general set is on the way.

CHAPTER 1

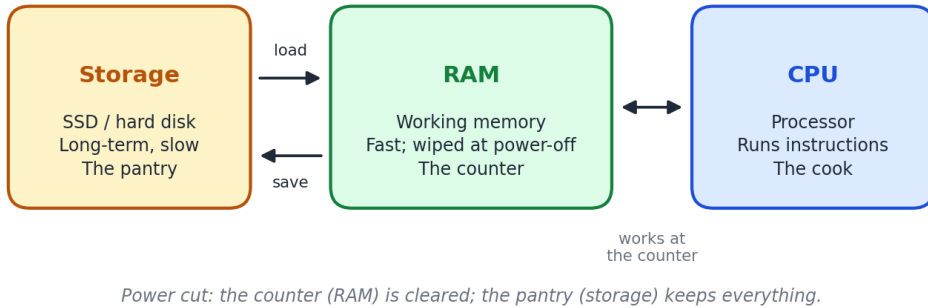
What a computer actually is

A computer is a machine that follows very simple instructions extremely fast. It does not understand anything. Everything it does, from playing a video to filtering your photos, is built from a handful of basic operations: move a number, add numbers, compare numbers, and jump to a different instruction depending on the result. Billions of times a second.

The three parts that matter

Think of a small restaurant kitchen with one cook.

PART	ITS JOB	KITCHEN ANALOGY
CPU	Runs instructions, billions per second	The cook
RAM	Holds what is being worked on right now; wiped at power-off	The counter
Storage	Keeps everything long-term; slower to reach	The pantry

Figure 1. The three parts that matter**Figure 1.** Storage, RAM and CPU, with the pantry, counter and cook analogy

When you open an app, it is copied from storage onto the counter (RAM) so the cook (CPU) can reach it quickly. When you "save" a file, you copy the result from the counter back into the pantry. This is why unsaved work disappears in a power cut: the counter is cleared, the pantry is not.

It also explains two things you have felt without naming. A laptop "slows down" with many apps open because the counter is full and the cook keeps walking to the pantry. And a phone that is "out of storage" has a full pantry: deleting an app frees pantry space, not counter space.

Everything is numbers

Inside the machine, everything is stored as **binary**: long sequences of 0s and 1s, called **bits**. Eight bits make a **byte**, and one byte can hold a number from 0 to 255.

Text, images, sound and programs are all just agreed ways of turning information into numbers. The letter A is stored as 65. A pixel in a photo is three numbers, for red, green and blue. A song is tens of thousands of numbers per second describing air pressure. A program is a sequence of numbers the CPU reads as instructions.

The meaning comes from **interpretation**, not from the bytes themselves. The same bytes could be read as text, as a picture, or as garbage, depending

on which program opens them. That is why opening a `.jpg` in Notepad shows nonsense: Notepad is reading picture-numbers as if they were letter-numbers.

Sizes you will see

- **KB** (kilobyte) $\approx$ 1,000 bytes: a text message, a short note
- **MB** (megabyte) $\approx$ 1 million bytes: a phone photo (2–5 MB), a song (3–10 MB), a PDF
- **GB** (gigabyte) $\approx$ 1 billion bytes: RAM is usually 8–32 GB; a film is a few GB; a phone has 64–512 GB of storage
- **TB** (terabyte) $\approx$ 1,000 GB: a large external drive, or a family's whole photo collection

A rule of thumb: a thousand photos take a few GB, so a 256 GB phone is not "full of photos"; it is usually full of videos and apps.

Try it

Open your computer's system information (Windows: *Settings* → *System* → *About*; Mac: *Apple menu* → *About This Mac*). Find three numbers: your processor name, your RAM in GB, and your storage size. Then right-click any photo and choose *Properties* (Mac: *Get Info*) and note its size in MB. How many of that photo would fit on the counter (RAM)? In the pantry?

Going deeper: what the cook actually does

The CPU runs a loop called **fetch, decode, execute**. Fetch the next instruction from RAM; work out what it means (add? compare? copy?); do it; repeat. A "3 GHz" processor runs this loop about three billion times a second. A "quad-core" processor is four cooks sharing one kitchen, which is why a modern laptop can encode a video and play music at once without either stalling.

Between the cook and the counter sits **cache**: a tiny, very fast shelf holding the few things the cook is using right this second. Between the counter and

the pantry, the gap is enormous: RAM is reached in nanoseconds, an SSD in microseconds, an old spinning hard disk in milliseconds. That is a factor of a thousand at each step, and it is why "more RAM" makes a computer feel faster even though the cook is no quicker: fewer trips to the pantry.

Two kinds of pantry: an **SSD** has no moving parts and is fast; a **hard disk** (HDD) is a spinning platter, cheaper per GB, and slow to find things. If a computer takes a minute to boot, it almost certainly has a hard disk.

One more thing you touch every day: when you press a key, the keyboard sends a number to the operating system (Chapter 2), which passes it to whichever app is in front. The app decides what the number means. That is why the same key types a letter in one window and moves a character in a game.

■ Watch alongside

Each link opens a YouTube search for the exact title; the first result is the video. The full series is the [Crash Course Computer Science playlist](#).

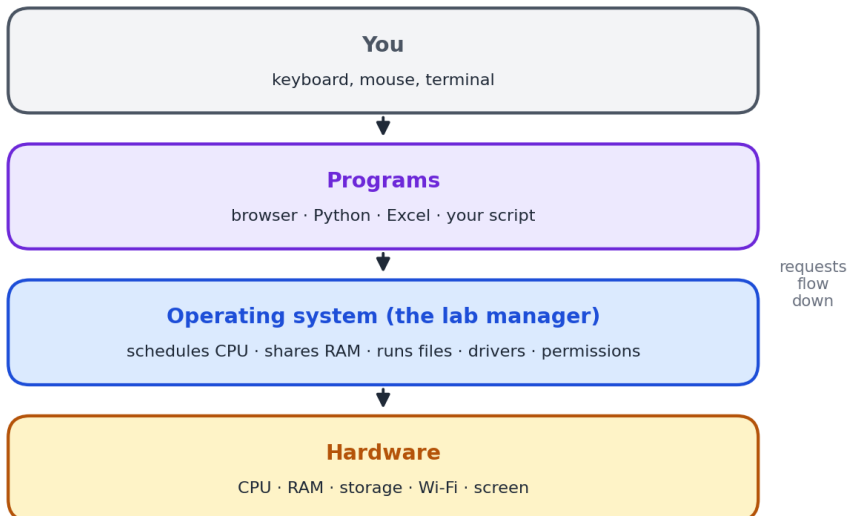
- [Crash Course CS #4: Representing Numbers and Letters with Binary](#) (11 min)
- [Crash Course CS #6: Registers and RAM](#) (12 min)
- [Crash Course CS #7: The Central Processing Unit](#) (12 min)
- [Crash Course CS #19: Memory and Storage](#) (12 min)

CHAPTER 2

The operating system

The **operating system (OS)** is the program that runs the computer on behalf of every other program. Windows, macOS, Linux, Android and iOS are all operating systems. You never really use the hardware directly; you use the OS, and the OS uses the hardware.

Figure 2. Everything goes through the operating system



A program never touches hardware directly: it asks the OS (a "system call").

Figure 2. You, programs, the operating system and hardware as layers, with requests flowing down

The building manager

If the CPU is the cook, the operating system is the manager of a large apartment building. Dozens of tenants (programs) want water, electricity, parking and the lift at once, and none of them may touch the boiler themselves. The manager:

- **Schedules time.** It switches the CPU between programs thousands of times a second, so your browser, a music player and a download all appear to run together. None of them knows it is being paused.
- **Allocates space.** It gives each program its own portion of RAM and stops programs from reading or overwriting each other's. A crashed game cannot corrupt your open document because the manager keeps their rooms separate.
- **Runs the storage.** It organises the pantry into files and folders (Chapter 3) and decides who may read or change each one.
- **Controls the devices.** Keyboard, screen, Wi-Fi, printer, camera and USB drives are all reached through the OS using small programs called **drivers**. "Windows can't find your printer" almost always means a missing or broken driver.
- **Checks permissions.** It knows which user is logged in and what they are allowed to do. That is why installing some software asks for an administrator password, and why an app on your phone asks before using the camera.

What the OS gives to programs

This part matters most for programming. A program cannot open a file, draw a window or connect to the internet on its own. It **asks the OS**, through a fixed set of requests called **system calls**. When a Python line says `open("notes.txt")`, Python fills in a request form and hands it to the OS, and the OS goes to the pantry.

This is also why a program built for Windows usually does not run on a Mac: the two building managers accept different request forms. The same program has to be rebuilt for each OS, which is why download pages offer a Windows version, a Mac version and sometimes a Linux one.

Programs and processes

- A **program** is the set of instructions sitting in storage, like a recipe written on a card.
- A **process** is that program actually running, with its own memory, like the recipe being cooked right now.

You can run the same program several times, giving several processes. A browser with ten tabs often runs ten or more processes, one per tab, so that one frozen page cannot freeze the rest. When you "force quit" something, you are asking the manager to end a process.

The families you will meet

OS	WHERE YOU SEE IT	NOTE FOR LATER
Windows	Most laptops and office PCs	Paths use \
macOS	Apple computers	Unix-based, like Linux
Linux	Servers, the cloud, Android underneath, Raspberry Pi	Most tutorials assume it
Android and iOS	Phones and tablets	Locked-down; apps live in sandboxes

Nearly every website you visit is served from a Linux computer, and "the cloud" (Chapter 12) is mostly rented Linux machines. That is why programming tutorials keep showing Linux commands even to Windows users.

Try it

Open **Task Manager** (Windows: `Ctrl + Shift + Esc`) or **Activity Monitor** (Mac: search in Spotlight). Look at the list of processes and the CPU and memory columns. Open a new browser tab and watch an entry appear; close it and watch it go. Sort by memory and find the hungriest process. You are looking directly at the building manager's ledger.

Going deeper: users, permissions and threads

Every file and every process belongs to a **user**. On your own laptop you are usually an *administrator*, allowed to install software and change system settings; a *standard* account can only work inside its own folders. Running as a standard user day to day is a real security measure: malware that runs as you can only do what you can do. The "Do you want to allow this app to make changes?" prompt is the OS stepping from one level to the other.

Inside a process there can be several **threads**: independent lines of work sharing the same memory. A messaging app has one thread drawing the screen and another waiting for the network, so typing never freezes while a photo downloads. Multiple cooks (cores, Chapter 1) can run threads truly at the same time.

The OS also runs many processes you never started: indexing your files for search, checking for updates, syncing a cloud folder. That is the "background processes" list in Task Manager, and why a laptop is busy even when you are doing nothing.

Watch alongside

- [Crash Course CS #18: Operating Systems \(14 min\)](#)
- [Crash Course CS #8: Instructions and Programs \(11 min\)](#)

CHAPTER 3

Files, folders and paths

A **file** is a named chunk of bytes in storage. A **folder** (also called a **directory**) is a container that holds files and other folders. Almost every beginner error of the "file not found" kind comes from not having a clear picture of this chapter.

The tree

Storage is organised as a **tree**, like a postal system: country → city → street → building → flat. At the very top is the **root**. On Windows each drive has its own root, such as `C:\`. On macOS and Linux there is a single root, written `/`.

```
C:\ (root)
├── Users
│   └── you
│       ├── Documents
│       │   ├── cv.docx
│       │   └── budget
│       │       └── expenses.csv
│       ├── Pictures
│       │   ├── 2026-06-goa
│       │   └── IMG_2041.jpg
│       └── Downloads
│           └── invoice.pdf
```

Paths: the address of a file

A **path** is the route through the tree to one file, written with separators between the levels.

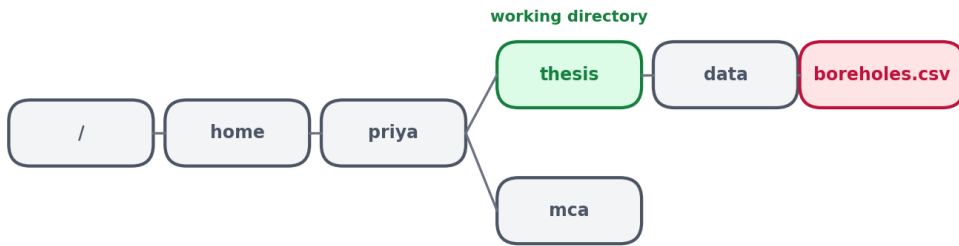
- Windows: `C:\Users\you\Documents\budget\expenses.csv`
- macOS / Linux: `/Users/you/Documents/budget/expenses.csv`

Windows separates levels with a backslash `\`; macOS and Linux use a forward slash `/`. Python accepts forward slashes on Windows too, which is why many tutorials use them everywhere.

There are two kinds of path, and the difference causes endless confusion:

- An **absolute path** starts at the root. It works from anywhere, like a full postal address with the country on it.
- A **relative path** starts from *where you currently are*, called the **working directory**. It is like saying "two doors down": correct only if you know which corridor you are standing in.

If your program is running in `C:\Users\you\Documents`, the relative path `budget\expenses.csv` finds the file. Run the same program from `C:\Users\you\Downloads` and it fails, even though nothing about the file changed. When you see `FileNotFoundError`, the first question is always: **where is my program running from?**

Figure 3. Absolute and relative paths

Absolute (from the root, works from anywhere):

`/home/priya/thesis/data/boreholes.csv`

Relative (from the working directory, here thesis):

`data/boreholes.csv`

Run the same code from mca and this relative path fails: `FileNotFoundError`

Figure 3. A folder tree showing the absolute path and the relative path from the Documents folder

Two special shortcuts appear everywhere:

- `.` means "the folder I am in"
- `..` means "the folder one level up"

And a third, on Mac and Linux: `~` means your home folder. PowerShell on Windows accepts `$HOME` for the same thing.

File extensions

The ending of a name, like `.jpg`, `.py` or `.pdf`, is the **extension**. It is only a label that tells the OS which program to open the file with. Renaming `holiday.jpg` to `holiday.pdf` does not turn a photo into a PDF, any more than relabelling a jar changes what is inside.

EXTENSION	WHAT IT IS
<code>.txt</code> , <code>.md</code>	Plain text, readable by any editor
<code>.csv</code>	A table stored as plain text, values separated by commas (Chapter 10)

EXTENSION	WHAT IT IS
<code>.py</code> , <code>.js</code> , <code>.java</code> , <code>.c</code>	Source code (plain text) in Python, JavaScript, Java, C
<code>.json</code>	Structured data as plain text, common for settings and web data
<code>.docx</code> , <code>.xlsx</code>	Word and Excel: zipped bundles of XML, not plain text
<code>.jpg</code> , <code>.png</code> , <code>.heic</code>	Photos and images; <code>.png</code> keeps sharp edges, <code>.jpg</code> compresses photos
<code>.mp3</code> , <code>.mp4</code> , <code>.mkv</code>	Audio and video
<code>.exe</code> (Windows), <code>.app</code> (Mac), <code>.apk</code> (Android)	Programs ready to run
<code>.zip</code>	A compressed bundle of other files

Notice how many of these are **plain text**. Code is just text in a file. Word documents are not plain text, which is why you must never write code in Microsoft Word: it will silently replace your quotation marks with curly ones and your program will not run. Windows hides extensions by default; turn them on (*View* → *Show* → *File name extensions*) or you will one day open `invoice.pdf.exe` thinking it is a PDF.

Hidden files

Files whose names start with a dot, such as `.gitignore` or `.env`, are hidden by default. Programming tools store settings in them. If a tutorial mentions a file you cannot see, turn on "show hidden files" in your file explorer.

Try it

In your file explorer, open any photo and find its full path. On Windows, click the address bar, or right-click the file and choose *Copy as path*; on Mac, right-click while holding Option and choose *Copy as Pathname*. Paste it somewhere and read it level by level from the root down. Then find your

Downloads folder's path and write, by hand, the relative path from Downloads to that photo, using `...`.

Going deeper: what "plain text" really means

A text file is bytes, and a rule for turning bytes into characters is called an **encoding**. The old rule, ASCII, covers 128 characters: English letters, digits, punctuation. The modern rule, **UTF-8**, covers every script in the world and emoji, using one byte for the old characters and two to four for the rest. Almost everything today is UTF-8. When you see `Ã©` where `é` should be, or `â€™` for an apostrophe, a file was written in one encoding and read in another. Python's `open(path, encoding="utf-8")` is how you say which rule to use.

The end of a line is also just a byte or two. Windows uses two (`\r\n`, carriage return and line feed); Mac and Linux use one (`\n`). Most tools cope, but it is why a file edited on Windows sometimes shows `^M` at the end of every line on a Linux server, and why Git may warn about "LF will be replaced by CRLF".

A file's name and its contents are stored separately. The name, size, dates and permissions live in the folder's index; deleting a file usually removes only the index entry, which is why deleted files can often be recovered until the space is reused, and why emptying the Recycle Bin is not the same as securely wiping.

■ Watch alongside

- [Crash Course CS #20: Files and File Systems](#) (12 min)
- [Crash Course CS #21: Compression](#) (13 min), what `.zip` and `.jpg` actually do

CHAPTER 4

The terminal

The **terminal** (also called the command line, shell, console or command prompt) is a way of giving the operating system instructions by typing instead of clicking. It looks old-fashioned, but it is the main working tool of programmers, because it is precise, repeatable and scriptable.

Why not just click?

Clicking is like ordering at a counter by pointing at the picture. Typing a command is like ordering by item number: less friendly, but exact, and it can be written down. A written command can be copied, shared, repeated a thousand times, and put in a tutorial. That is why every installation guide says "run this command" rather than "click here", and why people who automate things live in the terminal.

Reading a command

Every command has the same shape:

command	options	arguments
ls	-l	Pictures\2026-06-goa

- **command:** the program to run (ls means "list")

- **options** (or flags): adjust how it behaves, usually starting with `-` or `--` (`-l` means "long format, with details")
- **arguments**: what to act on (the folder `Pictures\2026-06-goa`)

The **prompt** is the text before your cursor, often ending in `$` or `>`. It usually shows your current working directory. In tutorials, `$ ls` means "type `ls`"; do not type the `$`.

The ten commands that cover most daily use

TASK	WINDOWS (POWERSHELL)	MACOS / LINUX
Where am I?	<code>pwd</code>	<code>pwd</code>
List files here	<code>ls</code> or <code>dir</code>	<code>ls</code>
Go into a folder	<code>cd Pictures</code>	<code>cd Pictures</code>
Go up one level	<code>cd ..</code>	<code>cd ..</code>
Make a folder	<code>mkdir sorted</code>	<code>mkdir sorted</code>
Show a text file	<code>cat notes.txt</code>	<code>cat notes.txt</code>
Copy a file	<code>cp a.jpg b.jpg</code>	<code>cp a.jpg b.jpg</code>
Move or rename	<code>mv a.jpg sorted\</code>	<code>mv a.jpg sorted/</code>
Delete a file	<code>rm old.jpg</code>	<code>rm old.jpg</code>
Run a Python file	<code>python script.py</code>	<code>python3 script.py</code>

Two warnings. First, `rm` deletes permanently; there is no Recycle Bin. Second, the terminal is silent when things succeed. No message usually means it worked.

Time-savers

- Press **Tab** to auto-complete file and folder names. This also prevents typos.
- Press the **up arrow** to bring back previous commands.

- Press **Ctrl + C** to stop a command that is running or stuck.
- Names with spaces need quotes: `cd "My Documents"`. Programmers avoid spaces in names for exactly this reason.

What actually happens when you type `python script.py`

The shell looks for a program called `python` in a list of folders called the **PATH** (an OS setting, not the same as a file path). If it finds one, it starts it as a new process and passes it `script.py` as an argument. Python then reads your file and runs it.

The common error `'python' is not recognized or command not found` means exactly one thing: the shell searched the PATH and did not find that program. Either it is not installed, or its folder was never added to the PATH. You will meet this at least once, and now you will know what it means.

Try it

Open a terminal (Windows: search for *PowerShell*; Mac: search for *Terminal*; Linux: `Ctrl + Alt + T`). Type `pwd`, then `ls`, then `cd` into Pictures or Downloads, and `ls` again. Make a folder called `sorted`, copy one photo into it with `cp`, and check with `ls sorted`. Then `cd ..` to come back. You have just done, by typing, what you usually do by dragging.

Going deeper: plumbing

Three small symbols turn the terminal from a way of running one command into a way of building tools.

Redirection. `>` sends a command's output into a file instead of the screen; `>>` appends instead of overwriting.

```
ls > files.txt           # a list of this folder, saved
echo "done" >> log.txt    # add a line to the end of log.txt
```

Pipes. `|` feeds one command's output into the next. On Mac and Linux:

```
ls | wc -l          # how many files are here? (wc -l counts lines)
cat notes.txt | grep milk  # only the lines containing "milk"
```

PowerShell has its own versions (`Measure-Object`, `Select-String`), but the idea is identical: small programs, each doing one thing, joined into a pipeline.

Environment variables. The shell keeps a set of named settings that every program it starts can read: `PATH` is one; `HOME` (your home folder) is another. `echo $env:PATH` in PowerShell, or `echo $PATH` on Mac and Linux, shows the list of folders searched for commands. When an installer offers to "add to `PATH`", it is editing this variable. Programs also read secrets from environment variables, such as an API key, so that the key never has to be written into the code.

Scripts. Any sequence of commands can be saved in a file (`.ps1` on Windows, `.sh` on Mac and Linux) and run as one. A script that renames all your holiday photos by date is fifteen minutes of learning away, and it is the first taste of what "automation" means.

■ Watch alongside

- [Crash Course CS #22: Keyboards and Command Line Interfaces](#) (11 min)
- [MIT Missing Semester, Lecture 1: Course Overview + The Shell](#) (about 50 min; notes on the [course site](#))
- Practise with the [Software Carpentry Unix Shell lesson](#)

CHAPTER 5

Setting up your computer

You have now met the operating system, paths and the terminal. Next, install three tools and check each one works: **Python** (the interpreter that runs code), **VS Code** (the editor you write code in) and **Jupyter** (notebooks, good for exploring data). Most beginners get stuck here, not on the code itself, so take your time and check each step.

Two words appear before their own chapters. The *interpreter* is the program that reads and runs your code; you'll meet it properly in Chapter 6. *pip* is Python's package installer; Chapter 7. For now, follow the steps like a recipe. Chapters 6–8 explain why each one works.

Step 1: Install Python

Windows. Go to python.org/downloads and download the latest Python 3 installer. Run it. On the **first screen**, tick "**Add python.exe to PATH**" at the bottom before clicking *Install Now*. This single checkbox prevents the most common beginner error (Chapter 4 explains PATH). When it finishes, close and reopen any open terminal windows.

Mac. Download the macOS installer from the same page and run it. Type `python3` and `pip3` wherever this book says `python` and `pip`.

Linux (Ubuntu and similar). Python is usually already installed. In a terminal, run `sudo apt install python3 python3-pip python3-venv`, then use `python3` and `pip3` as on a Mac.

Step 2: Check Python works

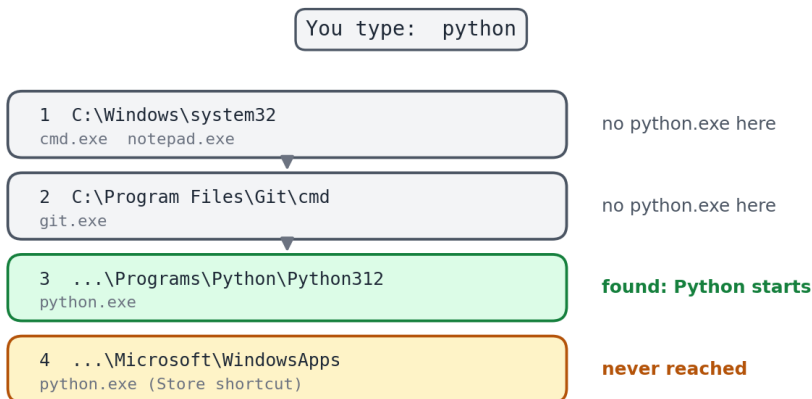
Open a terminal (Windows: press the Windows key, type *PowerShell*, press Enter) and type:

```
python --version
pip --version
```

You should see something like `Python 3.12.6` and a `pip` line ending in `(python 3.12)`. The exact numbers do not matter; seeing a version does.

If typing `python` opens the **Microsoft Store** instead, Windows is using a shortcut. Turn it off in *Settings* → *Apps* → *Advanced app settings* → *App execution aliases*, switching off both "python.exe" and "python3.exe", then open a new PowerShell window.

How the shell finds a command: it checks each PATH folder in order



Untick "Add python.exe to PATH" and folder 3 is missing: the search reaches folder 4 and opens the Microsoft Store, or ends with "'python' is not recognized".

Figure 9. How the shell searches the PATH folders in order and stops at the first `python.exe` it finds

Step 3: Install VS Code

1. Download it from code.visualstudio.com and install it with the default options (on Linux, the `.deb` or `.rpm` package).
2. Open VS Code, click the **Extensions** icon on the left (four small squares), and install **Python** and **Jupyter**, both published by Microsoft.

Step 4: Install the packages this book uses

In the terminal, run:

```
python -m pip install ipykernel pandas matplotlib requests
```

`ipykernel` lets Jupyter notebooks run Python. The others are introduced in Chapters 7, 9 and 15. The download takes a few minutes.

Step 5: Make a home for your work

Create a folder called `learning` inside your *Documents* folder. In VS Code choose **File** → **Open Folder** and open it. Always open a *folder* in VS Code, not single files. This keeps the working directory predictable (Chapter 3).

Step 6: Run your first program

1. In VS Code, create a new file called `hello.py` inside `learning`.
2. Type this line and save:

```
print("Hello. Python is working.")
```

3. Open VS Code's built-in terminal with **Terminal** → **New Terminal** and run:

```
python hello.py
```

If the message appears, your setup is complete.

If something goes wrong

YOU SEE	FIX
<code>'python'</code> is not recognized	PATH was not set. Re-run the installer, choose <i>Modify</i> , and tick "Add to environment variables". Then reopen the terminal.
The Microsoft Store opens	Turn off the app execution aliases (Step 2).
<code>pip</code> works but packages are "missing"	Always use <code>python -m pip install ...</code> (Chapter 8 explains why).
VS Code asks to "select interpreter"	Click it and choose the Python version you installed.
<code>python3:</code> command not found on Linux	Run the <code>apt install</code> line from Step 1.
Permission errors on Mac or Linux	Do not use <code>sudo pip</code> . Use a virtual environment (Chapter 8) instead.

Try it

Change the message in `hello.py` to include today's date and run it again. Then run `python --version` in VS Code's terminal and note the version number. Finally, run `python -c "print(7 * 6)"`: the `-c` option runs one line of code without any file.

Going deeper: what just got installed

The Python installer put three things on your computer: the interpreter (`python.exe`, one program), the **standard library** (hundreds of ready-made modules that come with Python; Chapter 7), and `pip`. It also wrote its folder into your PATH so the shell can find it. Everything you install with `pip` later goes into a folder called `site-packages` inside that Python's folder. Knowing this makes Chapter 8's puzzle, "I installed it but Python can't find it", obvious: there is more than one such folder on most computers.

VS Code is an **editor**, not Python. It could edit code in fifty languages, and without the interpreter it can run none of them. The Python extension you installed is what teaches it to underline mistakes, complete names and run files; it does that by talking to the interpreter you installed in Step 1.

■ Watch alongside

- [Corey Schafer: Python Tutorial for Beginners 1: Install and Setup for Mac and Windows](#) (about 20 min)
- [Corey Schafer: Visual Studio Code \(Windows\): Setting up a Python Development Environment](#) (watch the first half)

CHAPTER 6

What a programming language actually is

A programming language is a **precise written notation for instructions**, designed so that a human can write it and a program can translate it into the numbers a CPU understands. It is closer to a recipe written by a fussy chef, every quantity and step exact, than to a spoken language.

Why languages exist at all

The CPU only understands **machine code**: raw numbers meaning "load this", "add that", "jump there". Writing that by hand is possible but painfully slow. Programming languages let you write `total = price * 1.18` instead of dozens of numeric instructions. Something else then does the translation.

Two ways of translating

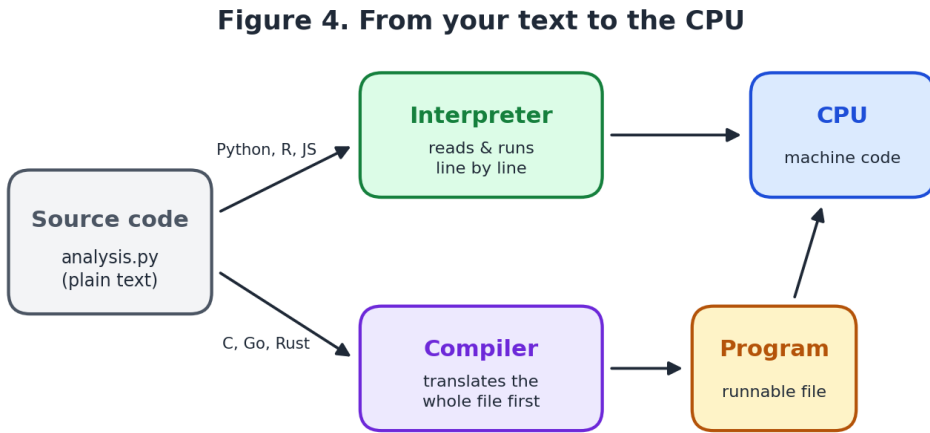


Figure 4. Source code goes either through an interpreter line by line, or through a compiler into a runnable program, before reaching the CPU

- **Compiled** (C, C++, Go, Rust): a **compiler** translates the whole program into machine code first, producing a runnable file. It is like translating a whole cookbook before printing it. There is an extra build step, errors show up early, and the result runs fast. The apps you download as `.exe` files were compiled.
- **Interpreted** (Python, JavaScript, R): an **interpreter** reads your code and carries it out line by line. It is like a live translator at a meeting. You can run code immediately, but an error only appears when that line is reached, and it runs more slowly.

Java sits in between: it compiles to an intermediate form that a virtual machine then runs. JavaScript, the language of web pages, is interpreted by your browser, which is why a web page can run code without installing anything.

The practical consequence: **when you "install Python", you are installing the interpreter.** Your `.py` file is just text. The `python` program is what reads it and makes it happen. No interpreter, no running code.

Source code, and where it runs

- **Source code** is the text you write, saved in files like `budget.py`.
- An **editor** or **IDE** (such as VS Code) is a program for writing that text comfortably. It is not the language. VS Code with no Python installed cannot run Python.
- The code runs **on a specific machine**, under that machine's OS, reading files from that machine's storage. When code "works on my laptop but not yours", the difference is almost always in that environment, not in the code.

Every language has the same core ideas

Once you see these, learning a second language becomes much easier. Each language just spells them differently.

IDEA	WHAT IT IS	EVERYDAY EXAMPLE
Variable	A named box holding a value	<code>price = 249</code>
Data type	What kind of value it is: number, text, true/false	"milk" is text; 2.5 is a number; <code>True</code> is a boolean
Collection	Many values together: a list or a table	A shopping list; a month of transactions
Condition	Do something only if a test is true	If the bill is over 500, flag it
Loop	Repeat an action	For each photo, add the date to its name
Function	A named, reusable procedure with inputs and outputs	<code>with_tax(price)</code>
Input / output	Reading and writing files, screen, network	Read <code>expenses.csv</code> , write <code>summary.csv</code>

The hard part of programming is rarely the syntax. It is breaking a problem into these small, exact steps. You already do this when you write directions for someone or plan a recipe for twelve.

Scripts and notebooks: two ways to write Python

So far, code lives in a **script**: a `.py` file you run from start to finish with `python budget.py`. A second format, the **Jupyter notebook** (a `.ipynb` file), is popular for exploring data.

A notebook is split into **cells**. Some cells hold code, others hold notes. You run one cell at a time, and its output, whether a number, a table or a chart, appears directly underneath, like a worksheet where every calculation sits next to its result.

Behind every notebook is a **kernel**: a running Python process that remembers everything the cells have done so far. This makes notebooks great for exploring step by step. It also causes their most common confusion: if you run cells out of order, or delete a cell, the kernel still remembers the old values. When a notebook behaves strangely, use **Restart Kernel and Run All** to replay it cleanly from the top.

USE A SCRIPT WHEN...	USE A NOTEBOOK WHEN...
The job runs start to finish	You are exploring or charting data
You will run it repeatedly, or on a schedule	You want notes, code and results together
Someone else needs to run it	You are following a data tutorial

To make a notebook in VS Code, create a file ending in `.ipynb`, choose your Python as the kernel when asked, type code in a cell and press **Shift + Enter** to run it.

■ Try it

Open a terminal and type `python` (or `python3`) and press Enter. You are now talking to the interpreter directly. Type `249 * 1.18` and press Enter. Then `item = "milk"` and then `item.upper()`. Then `len("milk")`. Type `exit()` to leave. You just ran code without any file at all.

Going deeper: types and structures

Every value has a **type**, and the type decides what you can do with it. `"5" + "5"` is `"55"` (joining text); `5 + 5` is `10`. The number one beginner error, `TypeError`, is mixing them: `"Total: " + 249` fails because Python won't guess whether you meant text or arithmetic. `str(249)` turns the number into text; `int("249")` turns text into a number. Data read from a file or a form always arrives as text, so this conversion is something you do constantly.

Beyond single values, languages give you a few ways of holding many:

- A **list** keeps things in order: `["milk", "bread", "eggs"]`. Positions count from 0, so `items[1]` is `"bread"`.
- A **dictionary** pairs keys with values: `{"milk": 62, "bread": 45}`. Look things up by name, not position. JSON (Chapter 9) is dictionaries and lists written down.
- A **set** holds each thing once; handy for "which categories appear?".
- A **table** (pandas' `DataFrame`) is a list of rows with named columns: a spreadsheet you can compute with.

Why this matters: most of what a program does is choose the right container. A month of expenses is a list of dictionaries, or better, a table. Once it is in the right shape, summing by category is one line.

■ Watch alongside

- [Crash Course CS #11: The First Programming Languages](#) (12 min)
- [Crash Course CS #12: Programming Basics: Statements and Functions](#) (12 min)
- [Harvard CS50P, Lecture 0: Functions, Variables](#) (free course; start with lecture 0, then 1 Conditionals and 2 Loops)

CHAPTER 7

Libraries, packages and package managers

A **library** is code someone else already wrote, packaged so you can reuse it. Almost no real program is written from scratch. Programmers spend much of their time finding, installing and combining libraries, then writing a small amount of their own code to glue them together.

The ready-made ingredients analogy

When you cook, you do not grind your own flour or make your own vinegar. You buy standard ingredients, check the label and the expiry date, and use them in your own recipe. Your contribution is the recipe and the judgement.

Libraries are the ready-made ingredients of programming:

- **Library / package:** the ingredient. A bundle of ready-made functions. ("Package" usually means a library bundled for installation. People use the words almost interchangeably.)
- **Package registry:** the supermarket. For Python it is **PyPI** (pypi.org); for JavaScript it is **npm**; for Rust, crates.io.
- **Package manager:** the delivery service that fetches the package, checks what else it needs, and installs it. For Python this is **pip**.
- **Version:** the batch number. Version 2.1 and version 3.0 of a library can behave differently.

- **Dependencies:** ingredients that your ingredient needs. Installing one package often quietly installs ten others.

The standard library vs. installed packages

Every language ships with a **standard library**: basic tools that come with the interpreter, like the salt and oil already in the kitchen. In Python, `math`, `csv`, `json`, `datetime`, `os` and `pathlib` are always there.

Everything else must be **installed** first. For everyday data work in Python, the common ones are:

PACKAGE	WHAT IT IS FOR
<code>pandas</code>	Tables of data: reading CSV and Excel, filtering, grouping
<code>matplotlib</code>	Plots and charts
<code>requests</code>	Fetching things from the internet
<code>openpyxl</code>	Reading and writing Excel files
<code>pillow</code>	Opening, resizing and converting images
<code>numpy</code>	Fast maths on arrays of numbers; pandas is built on it

The two steps: install, then import

This is the source of most beginner confusion, because the two steps happen in **different places**.

1. **Install** (once, in the terminal): download the package onto your computer.

```
pip install pandas
```

2. **Import** (every time, inside your code): tell *this program* you want to use it.

```
import pandas as pd
data = pd.read_csv("expenses.csv")
```

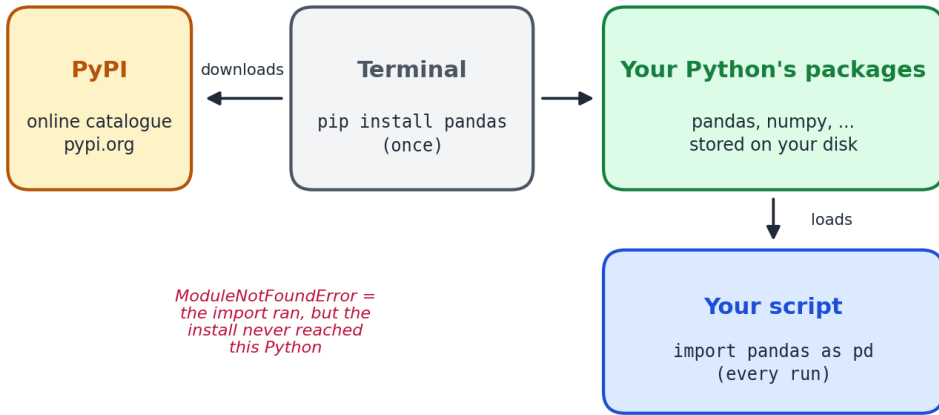
Figure 5. Install once, import every time

Figure 5. Pip downloads a package from PyPI into your Python's packages folder once; your script imports it every run

Installing is stocking the pantry. Importing is taking the jar down for today's recipe. If you see `ModuleNotFoundError: No module named 'pandas'`, the import happened but the install never did, or it was installed for a *different* Python (Chapter 8).

`pip install` is typed in the terminal, never inside a `.py` file or the Python `>>>` prompt. Typing it in the wrong place gives a `SyntaxError`.

How to judge a package

Anyone can publish a package, just as anyone can sell food online. Before using one, check:

- Is it widely used? (Download counts, GitHub stars, mentions in tutorials.)
- Is it maintained? (Recent releases, recent activity.)
- Does it have documentation with examples?
- Is the name spelled exactly right? Fake look-alike packages exist, and `pip install requests` will happily install something you did not want.

■ Try it

In the terminal, run `python -m pip install pandas`. Watch it download `pandas` and its dependencies. Then start `python`, type `import pandas`, and press Enter. No error means the ingredient is on your shelf. Now try `import pillow`: it fails, because the package is called `pillow` on PyPI but imported as `PIL`. The install name and the import name are not always the same; the documentation tells you.

Going deeper: the dependency tree

Run `python -m pip show pandas` and read the `Requires` line: `numpy, python-dateutil, pytz, tzdata`. Each of those has its own requirements. `pip` resolves the whole tree and installs it in the right order, which is why one `install` produced a dozen "Collecting..." lines. `python -m pip list` shows everything that ended up on the shelf.

This is also where trouble starts. Two packages can demand incompatible versions of a third; `pip` then reports a *conflict*. And a package you installed for one project can break another that wanted an older version. Chapter 8's answer is to give every project its own shelf.

One more idea you will meet: a **module** is one file of Python code; a **package** is a folder of modules. `import pandas` loads a package; `from pathlib import Path` loads one name from one module of the standard library. The standard library's `pathlib` is worth knowing early: `Path.home() / "Documents" / "budget"` builds a path that works on every operating system, without you thinking about slashes.

■ Watch alongside

- [Harvard CS50P, Lecture 4: Libraries](#) (`import`, the standard library and installing packages with `pip`)
- [Corey Schafer: Python Tutorial: pip](#) (about 15 min, practical walkthrough)

CHAPTER 8

Versions and environments

The same code can behave differently on two computers because the **environment** differs: which interpreter version, which packages, which versions of those packages, which OS. Controlling the environment is how programmers make results reproducible, the same way a recipe says "180 °C, fan oven" rather than "hot".

Version numbers

Most software uses three numbers: MAJOR.MINOR.PATCH, for example `pandas 2.2.1`.

- **PATCH** (2.2.1): bug fixes only. Safe to update.
- **MINOR** (2.2.1): new features, old code should still work.
- **MAJOR** (2.2.1): breaking changes. Code written for version 1 may fail on version 2.

You see the same scheme on your phone: an app going from 4.3.1 to 4.3.2 fixed something; going to 5.0 redesigned it. It matters when following tutorials. A tutorial from 2019 may use functions that no longer exist. When code from the internet fails in a strange way, check the date and the version it was written for.

"Which Python is this?"

Many computers end up with more than one Python installed: one from python.org, one from Anaconda, one bundled with another tool, one that came with the OS. Each has its **own** set of installed packages. This leads to the most frustrating beginner problem: you ran `pip install pandas`, it succeeded, but your code still says pandas is missing. The package went to one Python, and your code ran with another.

Two commands help you check:

```
python --version
pip --version
```

The second one shows which Python that pip belongs to. They should match. A safe habit is to install with `python -m pip install pandas`, which guarantees pip installs into the same Python you are running.

Virtual environments

A **virtual environment** is a separate, self-contained set of packages for one project. It is like giving each recipe its own small kitchen, so that the chilli for one dish can never end up in another.

Why bother? Project A may need `pandas 1.x` while Project B needs `pandas 2.x`. Without separate environments, installing one breaks the other. And when you share a project, the environment tells the other person exactly what to install.

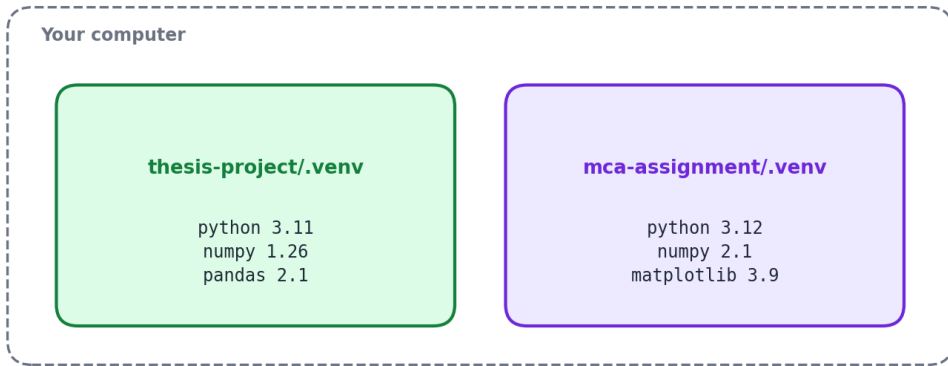
Figure 6. One sealed box of packages per project

Figure 6. Two projects on one computer, each with its own virtual environment and package versions

The basic Python workflow:

```
python -m venv .venv          # create the environment (once per
.venv\Scripts\activate        # Windows: switch it on
source .venv/bin/activate     # macOS/Linux: switch it on
python -m pip install pandas  # now installs only into this project
deactivate                    # switch it off
```

If Windows refuses to run the activate script with a message about "execution policies", run this once in PowerShell and try again: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

When an environment is active, its name usually appears in brackets at the start of your terminal prompt, such as `(.venv)`.

Recording the environment

Projects usually list their packages in a file named `requirements.txt`, which works like the ingredients list at the top of a recipe. Anyone can recreate your setup with:

```
pip install -r requirements.txt
```

And you make the list from a working environment with `python -m pip freeze > requirements.txt`.

Anaconda (`conda`) is another popular tool for the same job. It is common in scientific computing because it also handles non-Python libraries. Either approach is fine; just be consistent within a project.

■ Try it

Make a new folder for practice, `cd` into it, and create and activate a virtual environment using the commands above. Run `pip list` to see how empty it is: just `pip` itself. Install one package and run `pip list` again. Then `deactivate`, run `pip list` once more, and notice that the package is gone from view: it is still on disk, in the environment's own folder, but you are no longer standing in that kitchen.

Going deeper: pinning, lock files and containers

`requirements.txt` can say `pandas` (any version) or `pandas==2.2.1` (exactly this one). The first is convenient today and breaks in two years; the second reproduces exactly what you had. Serious projects **pin** versions, and many tools go a step further with a **lock file** that records every package in the whole dependency tree, down to the checksum of each download. You will see these as `poetry.lock`, `uv.lock` or, in the JavaScript world, `package-lock.json`. The idea is the same as `requirements.txt`, done thoroughly.

Sometimes even that is not enough: the code needs a particular OS library, or a specific version of the OS itself. The answer there is a **container** (Docker is the common tool); a whole miniature Linux system, with your program and everything it needs, packaged as one file that runs identically on any computer. It is the kitchen shipped with the recipe. When a tutorial says "just run the Docker image", that is what it means, and Chapter 12 has more on where such images run.

A rule of thumb for what to use: `venv` for anything on your own machine; a lock file when you share; a container when "works on my machine" is no longer an acceptable answer.

■ Watch alongside

- [Corey Schafer: VENV \(Windows\), How to Use Virtual Environments or the Mac and Linux version](#)
- [MIT Missing Semester, Lecture 8: Metaprogramming](#) (covers dependency management and versions; watch later)

CHAPTER 9

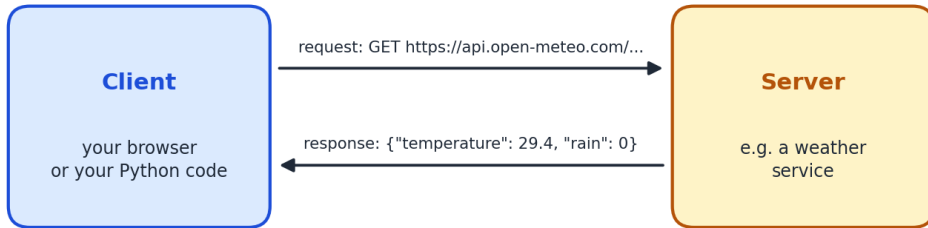
Networks, the internet and APIs

The internet is a huge network of computers that pass messages to each other using agreed rules called **protocols**. When you open a website, your computer sends a request message to another computer and gets a response back. Almost everything online, including apps, maps, payments and video, is this same request-and-response exchange.

Clients and servers

- A **client** asks for something. Your browser is a client; so is every app on your phone.
- A **server** is a program (and the computer it runs on) that waits for requests and answers them.

It is like ordering food through a delivery app. You (the client) send an order. The restaurant (the server) prepares it and sends it back. "The server is down" means the restaurant is closed; your order has nowhere to go. "Timed out" means you waited and no one answered.

Figure 7. Request and response

DNS first turns the name open-meteo.com into an IP address, like a phone directory.

Figure 7. A client sends a request to a server and receives a JSON response

Addresses

- Every computer on a network has an **IP address**, a number like `142.250.183.46`, like a street address.
- Humans use **domain names** like `wikipedia.org`. A system called **DNS** translates names into IP addresses, like the contacts list on your phone turns a name into a number. When "the internet is down" but the Wi-Fi icon looks fine, a DNS failure is a common cause.
- A **URL** is a full address for one resource: `https://api.open-meteo.com/v1/forecast?latitude=17.4` gives the protocol (`https`), the domain, the path (`/v1/forecast`) and, after the `?`, the **query**: named settings for this request. Notice the path looks just like a file path from Chapter 3.

`https` means the conversation is encrypted, so people in between, including whoever runs the café Wi-Fi, cannot read it. `http` without the `s` is not encrypted. Browsers now warn you about it.

Your home network

Your laptop, phone and TV share one Wi-Fi **router**. The router hands each device a private address (often starting `192.168.`) and presents one public address to the outside world, so the whole household looks like a single

customer to the internet. That is why two devices at home can talk to each other directly, and why "port forwarding" comes up when someone wants to reach a device at home from outside.

APIs: how programs ask each other for things

An **API** (Application Programming Interface) is a set of requests one program agrees to accept from another. A website is designed for humans to read. An API is designed for code to read.

When a weather app shows today's forecast, it did not read a weather website; it asked a weather API and received structured text, usually **JSON**:

```
{"temperature": 29.4, "rain": 0.0, "windspeed": 12.6}
```

Your code can then read that directly, with no manual downloading or copying from web pages. Payment, maps, translation, currency rates, train times: all of these are APIs that apps, and your own scripts, can call.

The word "API" is also used more broadly for *any* defined way to use a piece of code. A library's API is the list of functions it offers you. When documentation says "see the API reference", it means that list.

Local vs. remote

- **Local** means on your own computer. `localhost` (or `127.0.0.1`) is how a computer refers to itself.
- **Remote** means on another computer, such as a server, or "the cloud" (Chapter 12).

When a tutorial says "open `localhost:8000` in your browser", it means a server is running on your own machine, and your browser is acting as its client.

■ Try it

Paste this URL into your browser: `https://api.open-meteo.com/v1/forecast?latitude=17.38&longitude=78.49¤t=temperature_2m`. The page you see is raw JSON from an API, meant for code, not people. Find the current temperature in it. Change the latitude and longitude to your own city (search "coordinates of \") and reload. Later, you will fetch this same data with three lines of Python. If the address has changed since this was written, search for "Open-Meteo API".

Going deeper: what the request actually says

A request is a small text message. Its first line names a **method** and a path; the common methods are `GET` (give me this), `POST` (here is something new, such as a form you filled in), `PUT` (replace this) and `DELETE`. Then come **headers**: who is asking, what formats they accept, and, for anything private, a token proving who you are.

The response starts with a **status code**, a three-digit number you will learn to read at a glance:

CODE	MEANING
200	OK
301, 302	Moved; go to the new address
400	Your request was malformed
401, 403	You are not allowed (not logged in / no permission)
404	Not found
429	Too many requests; slow down
500, 502, 503	The server broke, or is overloaded, or is restarting

A **port** is a numbered door on a server: websites answer on 443 (https) and 80 (http); a server you run locally often picks 8000 or 3000; that is the number after the colon in `localhost:8000`.

Underneath, messages are cut into **packets**, each carrying a few hundred bytes, an address and a sequence number, and sent independently. They may take different routes and arrive out of order; the receiving computer reassembles them. This is why a download does not restart from scratch when one packet is lost, and why video calls glitch rather than stop.

And where do rate limits and API keys come from? A public API is a restaurant that anyone can order from, so it counts orders per customer. An **API key** is your customer number; keep it out of your code and in an environment variable (Chapter 4), and never post it anywhere public.

■ Watch alongside

- [Crash Course CS #28: Computer Networks](#) (12 min)
- [Crash Course CS #29: The Internet](#) (12 min)
- [Crash Course CS #30: The World Wide Web](#) (11 min)

CHAPTER 10

Data: spreadsheets, CSV, JSON and databases

Most of what you will ever want a computer to do is *something with a table*: a month of bank transactions, a list of contacts, a log of workouts, an inventory. This chapter is about the shapes data comes in, which shape to choose, and why the spreadsheet you already know is both the best and the worst tool for it.

Tidy tables

A table is rows and columns. It is **tidy** when each row is one thing (one transaction, one person, one day), each column is one property of that thing (date, amount, category), and each cell holds one value. Every tool in this book, and pandas above all, assumes tidy data.

The spreadsheets people actually make are rarely tidy: merged cells, a heading row halfway down, totals mixed in with data, colour used to mean something, two values in one cell ("Mumbai / Pune"). Humans read those fine; programs cannot. Ninety per cent of "data cleaning" is turning a human-friendly sheet into a tidy table.

DATE	MERCHANT	CATEGORY	AMOUNT
2026-09-02	Big Bazaar	groceries	1840.00

DATE	MERCHANT	CATEGORY	AMOUNT
2026-09-02	Metro	transport	45.00
2026-09-04	Netflix	subscriptions	649.00

Three habits make a table tidy from the start: dates as YYYY-MM-DD (they sort correctly and no program mistakes 03/04 for April or March), one unit per column (rupees, not "₹1,840"), and a header row with short names and no spaces (amount, not "Amount (Rs.)").

CSV: the table as plain text

CSV (comma-separated values) is the simplest possible table file: one row per line, commas between values, a header on the first line.

```
date,merchant,category,amount
2026-09-02,Big Bazaar,groceries,1840.00
2026-09-02,Metro,transport,45.00
```

It is plain text (Chapter 3), so every tool on every operating system can read it, it is tiny, and it works with Git (Chapter 14). Excel and Google Sheets both export to it with *Save as* or *Download*. What CSV cannot hold: formulas, formatting, multiple sheets, or a value that itself contains a comma without quoting it ("Smith, Anita"). Bank statements, exports from apps and open data portals nearly all arrive as CSV.

Spreadsheets: when they help and when they hurt

A spreadsheet is a table plus a calculator plus a chart tool, all in one window. For a hundred rows it is the right tool: look, sort, filter, total. Its weaknesses appear as the work grows: formulas hide inside cells so mistakes are invisible; a sort can silently detach a column from its rows; it happily turns 2/3 into a date and a phone number into 9.19E+11; and doing the same analysis next month means clicking through everything again.

The rule this book suggests: **spreadsheet to look, code to repeat**. When you find yourself doing the same steps a second time, that is the moment to export a CSV and write the steps down as a script (Chapter 15).

JSON: data with shape

Not everything is a flat table. A contact has several phone numbers; an order has many items; a weather forecast has hours inside days. **JSON** holds nested data: dictionaries (`{ }`, key-value pairs) and lists (`[ ]`), nested as deep as needed.

```
{
  "name": "Anita",
  "phones": ["+91 98...", "+91 99..."],
  "address": {"city": "Pune", "pin": "411001"}
}
```

JSON is what APIs return (Chapter 9), what apps use for settings files, and what a phone's exported data usually looks like. In Python it becomes dictionaries and lists directly: `data["address"]["city"]` reaches `"Pune"`. To analyse it as a table, you flatten it: one row per contact, one column per field, which pandas can do for you.

Databases: many tables that stay consistent

When data has several related tables (customers, orders, products), grows past what a file can hold, or is used by several people at once, it goes in a **database**. A database is a program that keeps tables on disk, enforces rules (every order must belong to an existing customer), answers questions fast even with millions of rows, and lets many users read and write without corrupting each other.

You ask it questions in **SQL**, a small language that reads almost like English:

```
SELECT category, SUM(amount) FROM expenses
WHERE date >= '2026-09-01'
GROUP BY category;
```

Every app on your phone has a database inside it. Most use **SQLite**, a whole database kept in one ordinary file, and Python includes it in the standard library (`import sqlite3`). Bigger systems use PostgreSQL or MySQL, which run as servers (Chapter 9) that apps connect to.

Which shape, when

SITUATION	USE
Looking at a few hundred rows once	A spreadsheet
Passing a table between programs or people	CSV
Nested or irregular data; talking to an API	JSON
Repeating an analysis every month	CSV in, a script, CSV or chart out
Several related tables, or many users	A database (start with SQLite)

Try it

Export something you already have as CSV: a bank or wallet statement, your phone's step count, or a Google Sheet. Open it in a plain text editor (Notepad, TextEdit), not a spreadsheet, and read the first three lines. Find the header, the separator, and one value that would confuse a program: a date written the ambiguous way, a currency symbol, a comma inside a name. That is your first data-cleaning list.

Going deeper: what a database does that a file cannot

Three things. **Indexes:** a database keeps sorted lookup tables so that finding one transaction among ten million takes milliseconds, not a scan of the whole file. **Transactions:** a set of changes either all happen or none do; when money moves between two accounts, the database guarantees you never end up with a half-finished transfer, even if the power fails halfway.

Concurrency: two people can add rows at the same instant without overwriting each other, something two people editing one CSV cannot manage.

The cost is that a database is not a file you can open and read. You need its tool, or a few lines of code, to look inside. For personal data, SQLite keeps that cost low: one file, no server, and a free viewer (search "DB Browser for SQLite") that shows it like a spreadsheet.

A last shape worth naming: **Parquet**, a compressed columnar file for large tables, which pandas reads and writes as easily as CSV. When a CSV passes a few hundred megabytes, Parquet is the usual next step.

■ Watch alongside

- [Corey Schafer: Python Tutorial: Working with JSON Data](#) (about 20 min)
- [Harvard CS50P, Lecture 6: File I/O](#) (reading and writing CSV)
- [Corey Schafer: Python SQLite Tutorial](#) (about 30 min, when you are ready for databases)

CHAPTER 11

Errors, and how to find answers

Professional programmers do not know everything. They are good at **reading errors and searching well**. They look things up dozens of times a day, and nobody considers that cheating. It is the job, just as a doctor consults references and a mechanic reads the manual.

Errors are data, not failure

An error message is the computer telling you exactly what went wrong and where. Beginners tend to panic and skip it. Instead, read it like a symptom: it is specific, and it is usually honest.

A Python error (called a **traceback**) looks like this:

```
Traceback (most recent call last):  
  File "budget.py", line 12, in <module>  
    per_day = total / days  
ZeroDivisionError: division by zero
```

Read it **from the bottom up**:

1. **Last line**: the type of error and a short description. This is the most important line.
2. **Lines above it**: where it happened, with the file name and line number.
3. **Further up**: the chain of calls that led there, often through library files you did not write. Useful later; ignore it at first.

Figure 8. Read a traceback from the bottom up

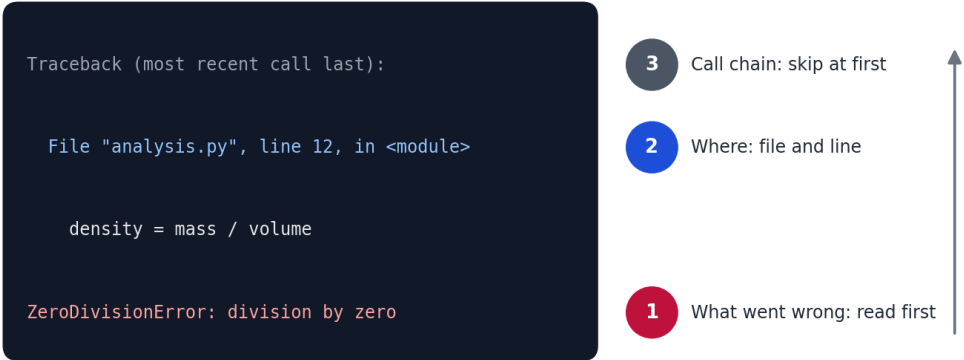


Figure 8. A Python traceback annotated to be read from the bottom line upwards

Some errors you will meet constantly:

ERROR	USUALLY MEANS
SyntaxError	You wrote something the language cannot parse: missing bracket, colon or quote
NameError	You used a name that was never defined, often a typo
TypeError	You combined incompatible kinds of value, such as adding text to a number
KeyError	You asked a dictionary or table for a column or key that isn't there; check the spelling and the header row
FileNotFoundError	The path is wrong, or you are running from a different folder (Chapter 3)
ModuleNotFoundError	The package is not installed in this Python (Chapters 7–8)
IndentationError	Spaces at the start of lines are inconsistent (Python cares about these)
PermissionError	The file is open in another program, or you are not allowed to write there

How to search: a method

1. **Copy the last line of the error exactly.** Paste it into a search engine.
2. **Add the language and key library.** For example: `python pandas KeyError: 'amount'`.
3. **Remove your own specific names** if results are poor. Your column name `amount` is unique to you; the error type is not. Try `pandas KeyError column`.
4. **Describe the goal, not the code**, when you do not know what to do at all: `python read excel file specific sheet, pandas group by and sum`.
5. **Check the date and version.** An answer from 2015 may be outdated (Chapter 8).

Searching for the *goal* is a skill worth practising. Almost every small task ("sort a table by a column", "rename files by date", "convert a PDF to text") has been asked and answered before.

Where good answers live

- **Official documentation** gives the definitive meaning of a function and its options. It can be dense, so jump to the "Examples" and "Getting started" sections.
- **Stack Overflow** is best for specific errors and "how do I..." questions. Prefer the accepted or highest-voted answer, and read the comments for updates.
- **Tutorials and blogs** are good for learning a topic end to end. Quality varies, so check the date.
- **AI assistants** can explain an error, a concept or a block of code, and are good at "what does this line do?". They can be confidently wrong, so test what they give you, and ask them to explain rather than only to fix. Never paste passwords, API keys or other people's personal data into one.

Reading documentation is like reading an appliance manual. You do not read it cover to cover. You go to the function you need, read its inputs and outputs, then look at the example.

Asking a person for help

When you ask a friend, a forum or a teacher, include:

- What you were **trying** to do
- The **exact** code (the smallest piece that shows the problem)
- The **full** error message, copied as text, not a phone photo of the screen
- What you already **tried**

Writing this out often reveals the answer before you even send it.

Programmers call this rubber-duck debugging: explain the problem to a rubber duck, and halfway through you hear it.

Try it

In Python, type `"Total: " + 249` and press Enter. Read the error bottom-up. Copy the last line into a search engine with the word `python`, open the first Stack Overflow result, and find the explanation. Then fix it yourself in two different ways.

Going deeper: finding the bug when there is no error

The hardest bugs produce no traceback: the program runs and gives the wrong answer. Two tools cover most cases.

Print debugging. Put `print()` statements at the points where you think you know what a value is, and check. `print(len(rows))`, `print(rows[0])`, `print(type(amount))`. Half the time, the value is not what you assumed: the amount is text, the list is empty, the date has a time attached. Remove the prints when done, or replace them with `logging`, the standard library's grown-up version that can be switched on and off.

The debugger. VS Code can run your program one line at a time. Click in the margin next to a line to set a *breakpoint*, press F5, and the program pauses there with every variable's current value shown in a panel. Step through with F10 and watch values change. It is the same thing as the print method, without editing the code. Python's built-in version is `breakpoint()`: put it on any line and the program stops there in the terminal.

A method for both: **narrow the problem**. Cut the data to three rows that still show the bug. Comment out half the program; does the bug remain? Keep halving. A bug you can reproduce in five lines is nearly always a bug you can see.

And the most common non-error error: the program ran, but not the file you edited. Check the path in the terminal, and whether you saved. It happens to everyone.

■ Watch alongside

- [Harvard CS50P, Lecture 3: Exceptions](#) (what errors are and how to handle them)
- [MIT Missing Semester, Lecture 7: Debugging and Profiling](#) (watch after a few weeks of coding)

CHAPTER 12

Staying safe

Everything in this book so far makes you more capable at the keyboard. This chapter is about not paying for it. None of it needs technical skill; all of it needs the context the earlier chapters gave you.

Passwords

The attack that gets most people is not clever. A website leaks its password list, and criminals try each email-and-password pair on every other site. If your email and your bank share a password, the bank is open. So:

- **A different password for every site.** Nobody can remember that, which is what a **password manager** is for: one program (Bitwarden, 1Password, or the one built into your browser or phone) that generates and stores a long random password for each site and fills it in for you. You remember one master password.
- **Length beats cleverness.** `correct horse battery staple` style passphrases of four or five random words are both stronger and easier to type than `P@ssw0rd!`.
- **Check whether you have already leaked.** The site haveibeenpwned.com tells you which breaches included your email. Change those passwords first.

Two-factor authentication

Two-factor authentication (2FA) means a second proof besides the password: a code from an app, a text message, or a hardware key. Even with your password, an attacker without your phone is stuck. Turn it on for email first (whoever controls your email can reset every other password), then banking, then everything that offers it. An authenticator app (Google Authenticator, Authy, or your password manager's built-in one) is safer than text messages, which can be intercepted by taking over your phone number.

Phishing

The second most common attack is being asked nicely. A message that looks like your bank, a courier, a tax office or a colleague, with a link and a reason to hurry: your account is locked, a parcel is waiting, the boss needs gift cards. The link goes to a copy of the real login page, and you type your password into it.

Habits that defeat almost all of it:

- **Urgency is the tell.** Real organisations do not need you to act in ten minutes.
- **Do not click; go directly.** Open the bank's app or type its address yourself.
- **Read the address, not the name.** `support@paypal.com` and `paypal.com.login-secure.xyz` are not PayPal. The domain is the part just before the first single `/`; everything before it can be anything.
- **Hover before clicking** on a computer; the real destination shows at the bottom of the window.
- **Attachments you did not ask for** stay closed. An invoice you were not expecting is an attack until proven otherwise.

The padlock, and what it does not mean

`https` and the padlock (Chapter 9) mean the connection between you and the site is encrypted: the café Wi-Fi cannot read it. They do **not** mean the

site is honest. Phishing sites have padlocks too. The padlock answers "can someone in between read this?", not "should I trust these people?".

Updates

Most successful attacks use holes that were fixed months earlier by an update the victim never installed. Turn on automatic updates for the operating system, the browser and the phone, and install them when asked. The ten-minute restart is the cheapest security you will ever buy. Software that no longer gets updates (an old phone, Windows 7, an abandoned app) is a door that no longer locks.

Backups

A laptop is stolen, a drive dies, a file is overwritten, ransomware encrypts everything. Backups are the difference between an annoying day and losing ten years of photos. The rule is **3-2-1**: three copies of anything that matters, on two different kinds of storage, one of them somewhere else. In practice: the original on your laptop, a copy on an external drive, and a copy in a cloud service. Then, the step everyone skips: **try restoring one file** to make sure the backup actually works.

Syncing (Google Drive, iCloud, OneDrive) is not quite a backup: delete a file on one device and the sync helpfully deletes it everywhere. It protects against a dead laptop, not against your own mistakes or ransomware. Most services keep deleted files for a while, which softens this; a true backup keeps old versions on purpose.

What the cloud actually is

"The cloud" is someone else's computers, rented by the hour. Google, Amazon and Microsoft run warehouses of Linux servers (Chapter 2); when you upload a photo to Google Photos, it lands on a disk in one of them. When a company "moves to the cloud", it stops owning servers and rents them instead. When you run something "in the cloud", you have rented a slice of a machine and are reaching it over the network (Chapter 9).

The practical consequences for you: your data is on a disk you cannot see, protected by that company's practices and your password; a service can shut down or change its terms; and anything you would not want a stranger to read should be encrypted before it leaves your machine.

■ Try it

Three tasks, twenty minutes. Check your email address at haveibeenpwned.com. Turn on two-factor authentication for your email account. Then open the last phishing-looking message in your inbox, without clicking anything, and read the sender's actual address and the link's real destination.

Going deeper: how the pieces fit

When you log in, the site does not store your password. It stores a **hash**: the result of a one-way scramble. Your typed password is scrambled the same way and the two scrambles compared. That is why a site can never tell you your old password, only reset it, and why a leak of a well-run site's database still needs to be cracked, one guess at a time. It is also why long passwords matter: every extra character multiplies the guesses needed.

Encryption comes in two flavours. **Symmetric** encryption uses one key to lock and unlock, like a house key; it is fast and is what protects the bulk of an `https` connection. **Public-key** encryption uses a pair: a public key anyone can use to lock a message, and a private key that alone can unlock it. That is how your browser can start a secret conversation with a site it has never met, and how a **digital signature** proves that an update really came from the company and not from someone in between. The padlock's certificate is a company's public key, vouched for by an authority your browser already trusts.

Lastly, the model to keep in your head: security is about **what the attacker has to do**, not about being unbreakable. A unique password plus 2FA means they need your password *and* your phone. Updates mean they need a hole nobody knows about yet. Backups mean that even if they win, they gain

nothing. Each layer makes you a worse target than the next person, and that is nearly always enough.

■ Watch alongside

- [Crash Course CS #31: Cybersecurity \(12 min\)](#)
- [Crash Course CS #32: Hackers and Cyber Attacks \(12 min\)](#)
- [Crash Course CS #33: Cryptography \(12 min\)](#)

CHAPTER 13

Learning without getting lost

The feeling of "I understand each line but not the point" comes from learning parts without a purpose to attach them to. The fix is to **learn through small, real problems** from your own life, and to learn only what each problem needs.

Principles

- **One language first.** A course may require C, Java or JavaScript, but pick one (Python is the gentlest) to build intuition. Concepts from Chapter 6 then transfer; only the spelling changes.
- **Type, don't copy.** Typing code, even from an example, forces you to notice each part. Copy-pasting teaches nothing, and it hides the one character you got wrong.
- **Change things and predict.** Before running a modified line, predict what will happen. When you are wrong, you have learned something specific.
- **Small steps, run often.** Write two or three lines, run them, check the result. Never write fifty lines and then run them for the first time.
- **Print everything.** When confused about what a variable holds, `print()` it. Looking beats guessing.
- **Being stuck is normal.** Spend 15–20 minutes trying (Chapter 11), then ask. Struggling a little builds understanding; struggling for hours just builds frustration.

- **Finish things.** A tiny working program teaches more than a big unfinished one, because only finished things get used, and only used things get improved.

A practice path from everyday life

Each step adds a few concepts. Do them in order, and use your own real files and numbers wherever you can.

1. **Calculator scripts.** Split a restaurant bill with tip among friends; convert a price between currencies; work out how many days until a date. (*Variables, numbers, arithmetic, `print`.*)
2. **Classify one thing.** Given an amount, print which budget band it falls in: small, medium, large. (*Conditions.*)
3. **Classify many things.** Put ten amounts in a list and classify each. (*Lists, loops.*)
4. **Make it reusable.** Turn the classifier into a function `band(amount)`. (*Functions.*)
5. **Read a real file.** Load a CSV export from your bank, wallet app or spreadsheet and classify every row. (*Files, paths, the `csv` module or `pandas`.*)
6. **Summarise.** Total per category; average per week; the biggest single expense. (*`pandas` grouping.*)
7. **Plot.** A bar chart of spending by category, or a line of daily totals. (*`matplotlib`.*)
8. **Fetch live data.** Ask a weather API (Chapter 9) for the next seven days and print which days need an umbrella. (*Packages, APIs, JSON.*)
9. **Automate a chore.** Rename a folder of photos by the date they were taken; combine twelve monthly CSVs into one; send yourself a summary. (*Loops over files, the standard library.*)

By step 9, every chapter of this book will have appeared in your own work, and you will have something you actually use.

Using your editor well

VS Code with its Python extension colours your code, underlines mistakes before you run them, and shows a function's documentation when you hover over it. Keep one folder per project, and open that folder in VS Code so the working directory is always predictable. Learn three shortcuts: `Ctrl + /` comments out a line, `F5` runs with the debugger, and `Ctrl + Shift + P` opens the command palette, from which everything else is reachable by name.

A weekly rhythm

- A few short sessions (30–60 minutes) beat one long weekend session.
- End each session by writing one line in a notes file: what you learned and what confused you. Next session, start from the confusion.
- Every few weeks, re-read a chapter of this book. It will mean more each time.
- When you finish something, however small, show it to someone. Explaining is the last step of understanding.

Try it

Do steps 1 and 2 today. Write `split.py`: a total, a tip percentage and a number of people, and print each person's share. Then `band.py`: an amount, and one of three words printed depending on its size. Run each, change a number, predict, run again.

Going deeper: how to read other people's code

Soon you will open a script someone else wrote, or one you wrote three months ago, and understand none of it. A method:

1. **Run it first.** Before reading a line, run it and see what it does. The output tells you what to look for.

2. **Find the entry point.** The bottom of the file, or a block starting `if __name__ == "__main__":`, is usually where things begin. Read from there, not from the top.
3. **Follow one value.** Pick one thing, the file name or the total, and trace where it goes. Ignore everything else on the first pass.
4. **Rename in your head.** `df`, `x`, `tmp` are habits, not information. Read them as what they hold.
5. **Change something small.** A print, a number, a colour. If the change shows up where you expected, your model is right.

The same method applies to tutorials and documentation. You are never expected to understand a whole file at once; nobody does. You are expected to find the part that matters and pull on it.

■ Watch alongside

- [Harvard CS50P, the full course](#): lectures 0–6 match practice steps 1–5 above; Lecture 6 (File I/O) covers reading CSV files
- [Crash Course CS #13: Intro to Algorithms](#) (12 min)
- [Automate the Boring Stuff with Python](#), free online, for step 9 and beyond

CHAPTER 14

Saving your history with Git

Git records snapshots of a project folder over time, so you can see what changed, when and why, and go back to any earlier version. It replaces the familiar mess of `report_final.docx`, `report_final2.docx` and `report_REALLY_final.docx`. Almost every programming project in the world uses it, and it works for any folder of text: code, notes, a novel, a recipe collection.

Track changes for a folder

You may know *Track Changes* in Word: every edit is recorded and can be accepted or undone. Git is that, for a whole folder, with a note attached to each batch of changes saying why you made them.

- A **repository** (repo) is a folder whose history Git is tracking.
- A **commit** is one dated entry: a snapshot of chosen files plus a short message saying why.
- The **log** is the list of all commits, newest first.
- **Staging** is choosing which changes go into the next entry, like picking which edits to accept before saving a version.
- **GitHub** is a website that stores a copy of your repository online, for backup and sharing. Git works fine without it.

Install and introduce yourself

Download Git from git-scm.com and install it with the default options (on Mac it comes with the developer tools; on Ubuntu, `sudo apt install git`). Then, in a new terminal, tell Git who you are (once per computer):

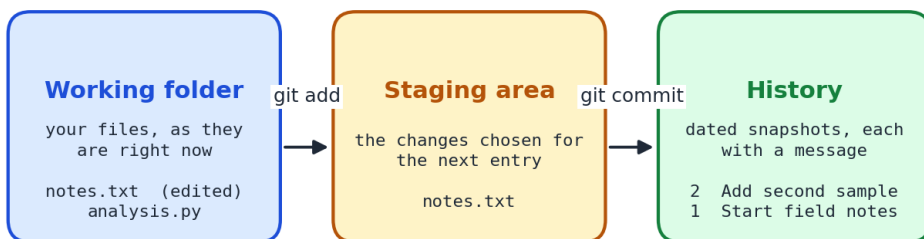
```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

The everyday cycle

```
cd C:\Users\you\Documents\learning
git init                        # start tracking this folder (once)
git status                     # what has changed?
git add split.py               # stage a file for the next commit
git commit -m "Bill splitter"  # save the snapshot with a message
git log --oneline              # see the history
```

From then on, the rhythm is: edit files → `git status` → `git add` → `git commit -m "why"`. Commit whenever you reach a small working step. Write messages that explain *why*, such as "Handle tips in percent", not "changes".

Git's three trays: working folder, staging area, history



`git status` reports what is in each tray. `git log` lists the history, newest first.

Figure 10. Git's three trays, the working folder, the staging area and the history, with `git add` and `git commit` moving changes between them

Things Git should ignore

Some files should never be committed: virtual environments, large data or video files, passwords and API keys, and the `.DS_Store` and `Thumbs.db` files operating systems scatter about. List them in a plain-text file named `.gitignore` in the repository folder:

```
.venv/  
*.mp4  
.env  
.DS_Store
```

Using Git from VS Code

VS Code has Git built in. The **Source Control** icon on the left (a branching line) shows changed files. Type a message in the box and click **Commit**. It runs exactly the commands above, so knowing them helps you understand what the button does, and what to do when it complains.

Try it

In your `learning` folder, run `git init`, then commit `split.py`. Change the tip percentage in `split.py`, run `git status` to see Git notice the change, and make a second commit. Finally run `git log --oneline` and count your commits. Then run `git diff HEAD~1` to see exactly which lines changed between the two.

Going deeper: branches, remotes and conflicts

A **branch** is a parallel line of history. `git switch -c try-percent` starts one; you commit on it as usual; the original line, usually called `main`, is untouched. If the experiment works, `git switch main` and `git merge try-percent` fold it in; if not, delete the branch and nothing happened. Branches are how you try something risky without wrecking what works, and how several people work on one project at once.

A **remote** is a copy of the repository somewhere else, usually GitHub. `git push` sends your commits there; `git pull` fetches commits others (or you, from another computer) added. The remote is also the backup Chapter 12 asked for: a stolen laptop costs you nothing committed and pushed. Creating a GitHub account and pushing one project is a good first weekend task; GitHub's own "Hello World" guide covers it.

A **merge conflict** happens when two branches changed the same lines differently. Git stops and marks the spot in the file:

```
<<<<<< HEAD
tip = 0.10
=====
tip = 0.15
>>>>>> try-percent
```

You pick one (or write a third), delete the markers, and commit. It looks alarming the first time and is routine by the third.

What Git is bad at: large binary files (photos, video, Word documents), which it stores whole on every change, and secrets, which stay in history even after you delete them. Keep data and media out with `.gitignore`, and if a password ever gets committed, treat it as leaked and change it.

■ Watch alongside

- [MIT Missing Semester, Lecture 6: Version Control \(git\)](#) (about 85 min; the first 30 minutes are enough to start)
- Practise with the [Software Carpentry Version Control with Git](#) lesson
- Read chapters 1–3 of [Pro Git](#) (free); chapter 3 is branches

CHAPTER 15

Capstone: a month of expenses, end to end

This chapter builds one small project from an empty folder to a finished chart, and uses almost every idea in the book along the way. You will read a month of expenses, put each one in a size band, total by category, plot it, save the results, and record the work in Git. Type the code yourself rather than copying it (Chapter 13).

The data below is invented. Once it works, run the same script on a real export from your bank, wallet app or spreadsheet; the only change needed is the column names.

Step 1: Set up the project (Chapters 3, 5, 8, 14)

In the terminal:

```
cd C:\Users\you\Documents\learning
mkdir budget
cd budget
python -m venv .venv
.venv\Scripts\activate
python -m pip install pandas matplotlib requests
git init
```

(Mac and Linux: `source .venv/bin/activate`.) Then open the `budget` folder in VS Code (*File* → *Open Folder*). You now have a project folder, a private set of packages, and an empty history.

Step 2: Create the data file (Chapters 3, 10)

In VS Code, create `expenses.csv` in the `budget` folder and paste in this table. Amounts are in rupees; use whatever currency you like, the code does not care.

```
date,merchant,category,amount
2026-09-01,Metro card,transport,500.00
2026-09-01,Ratnadeep,groceries,1840.00
2026-09-02,Chai point,eating_out,60.00
2026-09-03,Netflix,subscriptions,649.00
2026-09-04,Uber,transport,245.00
2026-09-05,Apollo Pharmacy,health,320.00
2026-09-06,Paradise,eating_out,780.00
2026-09-07,Electricity bill,bills,2150.00
2026-09-08,Ratnadeep,groceries,960.00
2026-09-09,Swiggy,eating_out,410.00
2026-09-10,Spotify,subscriptions,119.00
2026-09-11,Auto,transport,80.00
2026-09-12,Amazon,shopping,1499.00
2026-09-13,Chai point,eating_out,45.00
2026-09-14,Ratnadeep,groceries,1230.00
2026-09-15,Mobile recharge,bills,299.00
2026-09-16,Uber,transport,310.00
2026-09-18,Swiggy,eating_out,365.00
2026-09-19,Decathlon,shopping,2299.00
2026-09-21,Ratnadeep,groceries,1120.00
2026-09-22,Auto,transport,70.00
2026-09-24,Internet bill,bills,999.00
2026-09-26,Apollo Pharmacy,health,180.00
2026-09-28,Ratnadeep,groceries,875.00
```

The first line is the **header**: column names. Every later line is one transaction. It is a tidy table (Chapter 10): one row per thing, one column per property, dates written `YYYY-MM-DD`.

Step 3: Read the data (Chapters 6, 7)

Create `budget.py` and start with:

```
import pandas as pd
import matplotlib.pyplot as plt

expenses = pd.read_csv("expenses.csv", parse_dates=["date"])
print(expenses.head())
print("Rows and columns:", expenses.shape)
```

- `import pandas as pd` loads the installed pandas package and gives it the short name `pd`.
- `pd.read_csv("expenses.csv")` reads the file into a **DataFrame**, pandas' name for a table. The path is **relative**, so this works only when you run from the `budget` folder. `parse_dates` tells pandas the first column is dates, not text.
- `expenses.head()` shows the first five rows; `expenses.shape` gives (rows, columns).

Run `python budget.py`. You should see five rows, then Rows and columns: (24, 4).

Step 4: Classify one value (Chapter 6)

Add a **function** that turns an amount into a size band:

```
def band(amount):
    if amount < 200:
        return "small"
    elif amount < 1000:
        return "medium"
    else:
        return "large"
```

`def` defines the function; `amount` is its input. The `if` / `elif` / `else` **conditions** are checked from top to bottom, and the first true one decides the answer. Indentation (four spaces) shows which lines belong inside the function and inside each condition.

Step 5: Classify every transaction (Chapters 6, 7)

```
expenses["band"] = expenses["amount"].apply(band)
print(expenses["band"].value_counts())
```

`expenses["amount"]` picks one column. `.apply(band)` runs your function on every value in it, which is a **loop** without writing the loop yourself. The results go into a new column called `band`, and `value_counts()` counts how many of each.

Step 6: Summarise (Chapters 7, 10)

```
summary = expenses.groupby("category").agg(
    count=("amount", "count"),
    total=("amount", "sum"),
).sort_values("total", ascending=False)
print(summary)
print("Total spent:", expenses["amount"].sum())
print(expenses.sort_values("amount", ascending=False).head(3)[["date",
"merchant", "amount"]])
```

`groupby` sorts rows into piles by category, like sorting receipts into envelopes. `agg` then counts each pile and totals it. `sort_values` puts the biggest category first. Check that the result makes sense: groceries and bills should dominate a normal month, and the total should match what you actually spent.

Step 7: Plot and save (Chapters 3, 7)

```
fig, ax = plt.subplots()
summary["total"].plot.bar(ax=ax)
ax.set_ylabel("Total (₹)")
ax.set_title("September spending by category")
fig.tight_layout()
fig.savefig("by_category.png", dpi=150)

expenses.to_csv("expenses_classified.csv", index=False)
print("Saved expenses_classified.csv and by_category.png")
```

`plot.bar` draws one bar per category. `savefig` and `to_csv` write new files into the working directory; open the `budget` folder and you will find them there.

Step 8: Record the work (Chapter 14)

Create a `.gitignore` file containing `.venv/`, then:

```
git add .gitignore expenses.csv budget.py
git commit -m "Classify a month of expenses and plot by category"
```

The outputs (`expenses_classified.csv`, the PNG) can always be regenerated by running the script, so many people leave them out of Git.

Step 9 (extension): live weather from an API (Chapter 9)

Create `umbrella.py`. It asks the Open-Meteo API for the next seven days and prints which ones need an umbrella:

```
import requests

url = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": 17.38,
    "longitude": 78.49,
    "daily": "temperature_2m_max,precipitation_probability_max",
    "timezone": "auto",
}
data = requests.get(url, params=params, timeout=30).json()

days = data["daily"]["time"]
highs = data["daily"]["temperature_2m_max"]
rain = data["daily"]["precipitation_probability_max"]

for day, high, chance in zip(days, highs, rain):
    note = "umbrella" if chance >= 50 else ""
    print(day, f"{high} °C", f"{chance}% rain", note)
```

`requests.get` is the **client** sending a request (Figure 7); `params` becomes the part of the URL after the `?`. `.json()` turns the JSON reply into Python dictionaries and lists. The loop walks three parallel lists at once. Change the

coordinates to your own city. Because the data is live, your output will differ every day; if the address has changed since this was written, search for "Open-Meteo API" and adjust the URL.

■ Try it

Change the bands so that anything under 500 is "small" and anything 2,000 or more is "large". Run the script again, check the new counts, and commit the change with a clear message. Then swap in a real export of your own spending and see what the chart says about you.

■ Watch alongside

- [Harvard CS50P, Lecture 6: File I/O \(reading and writing files, including CSV\)](#)
- [Corey Schafer: Pandas Tutorial \(Part 1\): Getting Started with Data Analysis and the next few parts](#)
- [Corey Schafer: Matplotlib Tutorial \(Part 2\): Bar Charts](#)

Afterword: your own project

The capstone used invented expenses. The next step is the same project with something you care about, in your own folder, on your own machine. Here is the capstone as a template; the blanks are yours to fill.

1. **Name the question.** Write one sentence: *I want to know _____ about _____*. If you cannot write it, the project is not ready yet.
2. **Get the data into a CSV.** One row per thing; one column per property; a header row; dates as YYYY-MM-DD; no merged cells, no colour coding, no totals row. From Excel or Sheets, *Save As* → CSV. From an app, look for *Export*.
3. **Make the folder.** Documents\projects_____ containing data\ for the CSV, analysis.py, a .venv, a .gitignore with .venv/ in it, and later requirements.txt.
4. **Read it and look.** `print(data.shape)`, `print(data.head())`, `print(data.dtypes)`. A column that should be numbers but shows `object` has a stray text value somewhere; find it before going on.
5. **Compute one thing.** A function, like `band`, that turns one row into one answer. Test it with three values whose answers you already know.
6. **Apply, group, summarise, plot.** `apply`, `groupby`, `agg`, one figure saved to a PNG.
7. **Commit** the CSV and the code, not the outputs.
8. **Write a README.md:** the question, where the data came from, and the one command that runs the analysis.

Six projects that fit this template:

- **Photos by month.** A script that walks your photo folder, reads each file's date, and moves it into `2026\09\`. (Files, loops, the `pathlib` and `shutil` modules.)
- **Subscriptions audit.** Every recurring charge from a year of statements, with the annual total. Most people find one they forgot.
- **Step count or sleep.** Export from your phone; average by weekday; plot a year. Which day of the week are you laziest?
- **Group chat statistics.** Export a chat; count messages per person and per hour; find the most-used word. (Text handling, dictionaries.)
- **Grocery price tracker.** Ten items, one row per shop visit; a line per item over time. (A table that grows, dates, plotting several series.)
- **Umbrella bot.** The weather script from Chapter 15, run every morning by the operating system's scheduler (Task Scheduler on Windows, `cron` on Mac and Linux), writing one line to a log file. (Automation, the terminal, APIs.)

When the first one works, you are no longer doing exercises.

Appendix: What you should see

Check your "Try it" results here *after* attempting each one. Your exact numbers may differ from these; what matters is the shape of the result.

Chapter 1. A processor name (for example *Intel Core i5*, *AMD Ryzen 5* or *Apple M2*), RAM of roughly 8–32 GB, and storage of roughly 256 GB–1 TB. A phone photo is 2–5 MB, so a few thousand fit in RAM and a hundred thousand in storage.

Chapter 2. A long list of processes, far more than the programs you opened, because the operating system runs many background processes. A new browser tab adds one or more entries; the hungriest process is usually the browser.

Chapter 3. A path such as `C:\Users\you\Pictures\2026-06-goa\IMG_2041.jpg`. Reading it from the left: drive root, then Users, your folder, Pictures, the album, the file. The relative path from Downloads is `..\Pictures\2026-06-goa\IMG_2041.jpg`: up one level to your user folder, then down.

Chapter 4. `pwd` prints your current folder, usually `C:\Users\you` when PowerShell first opens. After `cd` into a folder, `pwd` shows that folder added on the end. `ls sorted` lists the one photo you copied. After `cd ..` you are back where you started.

Chapter 5. Your new message prints in the terminal. `python --version` shows `Python 3.` followed by two more numbers, such as `Python 3.12.6`. `python -c "print(7 * 6)"` prints 42.

Chapter 6. `249 * 1.18` gives `293.82`. `item.upper()` gives `'MILK'`. `len("milk")` gives `4`. The prompt `>>>` means the interpreter is waiting for your next line.

Chapter 7. `pip` downloads `pandas` plus several dependencies, such as `numpy` and `python-dateutil`, then ends with "Successfully installed...". `import pandas` prints nothing at all; silence after an import means success. `import pillow` fails with `ModuleNotFoundError`; the package is imported as `PIL`.

Chapter 8. Right after creating the environment, `pip list` shows only `pip` itself (and perhaps `setuptools`). After installing one package, it shows that package plus its dependencies. After `deactivate`, `pip list` shows your main Python's packages instead, and the prompt loses its `(.venv)` prefix.

Chapter 9. The browser shows raw text starting with `{"latitude": 17.375, "longitude": 78.5, ...}`. Inside `"current"` there is a `"temperature_2m"` number. Changing the coordinates changes the numbers.

Chapter 10. The first line of the export is the header. Common traps: a date like `02/09/2026` (which is it?), an amount like `₹1,840.00` (the symbol and the comma both stop it being a number), a merchant name containing a comma, and blank lines at the top or bottom.

Chapter 11. The last line reads `TypeError: can only concatenate str (not "int") to str`. Python will not join text and a number. Fix it with `"Total: " + str(249)`, or with an f-string: `f"Total: {249}"`.

Chapter 12. `haveibeenpwned.com` either reports no breaches or lists the sites that leaked your address; change those passwords first. Two-factor setup ends with a backup-codes screen: save those codes somewhere safe. The phishing message's real sender address and link destination will not match the organisation it claims to be from.

Chapter 13. `split.py` prints one number, the share per person: for a 1,200 bill, a 10% tip and 3 friends, `440.0` ($1,200 \times 1.10 \div 3$). If your number differs, check the order of operations; when you and the program disagree, the program is usually right and an assumption is wrong. `band.py` prints one of three words.

Chapter 14. `git status` shows `split.py` as *modified* in red after your edit. `git log --oneline` lists two commits, newest first, each with a short code and

your message. `git diff HEAD~1` shows the old tip line with a `-` and the new one with a `+`.

Chapter 15. With the practice data: 24 rows and 4 columns; bands of 12 medium, 6 large and 6 small; groceries on top of the summary at 6,025.00 and health at the bottom at 500.00; a total of 17,404.00; the biggest three are Decathlon (2,299), the electricity bill (2,150) and the first grocery shop (1,840). After the Try it change the bands become 12 small, 10 medium and 2 large. The umbrella script prints seven dated lines; the numbers change daily.

Cheat sheet: the commands you will keep looking up

Terminal (Chapter 4)

COMMAND	WHAT IT DOES
<code>pwd</code>	Show the folder you are in
<code>ls</code>	List what is here
<code>cd folder</code> · <code>cd ..</code> · <code>cd ~</code> (<code>cd \$HOME</code> on Windows)	Go into a folder · up one level · home
<code>mkdir name</code>	Make a folder
<code>echo text > file.txt</code> · <code>echo text >> file.txt</code>	Create or overwrite a file · add a line to it
<code>cat file.txt</code>	Show a file's contents
<code>cp a b</code> · <code>mv a b</code> · <code>rm a</code>	Copy · move or rename · delete (no Recycle Bin)
<code>ls wc -l</code> (the bar is typed as a vertical pipe character)	Feed one command's output into another (Mac and Linux)
Tab · ↑ · Ctrl + C	Complete a name · repeat an earlier command · stop a command

Python and pip (Chapters 5–7)

COMMAND	WHAT IT DOES
<code>python --version</code>	Which Python answers to <code>python</code>
<code>python script.py</code>	Run a script from the folder you are in
<code>python -c "print(2 + 2)"</code>	Run one line without making a file
<code>python ... exit()</code>	Start and leave the interactive <code>>>></code> prompt
<code>python -m pip install pkg</code>	Install a package into this Python
<code>python -m pip show pkg · python -m pip list</code>	Details of one package · all of them
<code>python -m pip freeze > requirements.txt</code>	Record the installed packages
<code>python -m pip install -r requirements.txt</code>	Install from a recorded list

Virtual environments (Chapter 8)

COMMAND	WHAT IT DOES
<code>python -m venv .venv</code>	Create an environment in this folder
<code>.venv\Scripts\activate · source .venv/bin/activate</code>	Turn it on (Windows · Mac and Linux); the prompt shows <code>(.venv)</code>
<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>	Once, if Windows refuses to run the activate script
<code>deactivate</code>	Turn it off

Git (Chapter 14)

COMMAND	WHAT IT DOES
<code>git config --global user.name "..." · user.email "..."</code>	Once, before the first commit
<code>git init</code>	Start tracking this folder
<code>git status</code>	What is changed, staged and untracked

COMMAND	WHAT IT DOES
<code>git add file · git add .</code>	Stage one file · everything
<code>git commit -m "Why this change"</code>	Save a snapshot with a message
<code>git log --oneline · git diff</code>	The history, newest first · what changed since the last commit
<code>git switch -c name · git switch main · git merge name</code>	Start a branch · go back · fold it in
<code>git push · git pull</code>	Send commits to the remote · fetch new ones
<code>.gitignore</code> containing <code>.venv/</code>	Keep the environment out of Git

pandas, the six lines (Chapters 10, 15)

LINE	WHAT IT DOES
<code>df = pd.read_csv("file.csv")</code>	Read a table
<code>df.head() · df.shape · df.dtypes</code>	Look at it
<code>df["col"] · df[df["amount"] > 500]</code>	One column · the rows that match
<code>df["new"] = df["col"].apply(f)</code>	A new column from a function
<code>df.groupby("cat").agg(total=("amount", "sum"))</code>	Totals per group
<code>df.to_csv("out.csv", index=False)</code>	Write a table

Reading an error (Chapter 11). Last line first. `ModuleNotFoundError`: install it into this Python. `FileNotFoundError`: which folder am I running from? `NameError`: a spelling mismatch. `KeyError`: that column or key does not exist; check the header. `TypeError` with `str`: text and a number joined with `+`. `SyntaxError`: look at the line above the arrow, often a missing colon or bracket.

Glossary

TERM	PLAIN MEANING	CHAPTER
API	A defined set of requests a program or library accepts from other code	9
API key	Your customer number for an API; keep it out of your code	9
Absolute path	A file address starting from the root	3
Argument	Something passed to a command or function to act on	4, 6
Backup (3-2-1)	Three copies, two kinds of storage, one elsewhere	12
Binary	Numbers written with only 0 and 1; how computers store everything	1
Branch	A parallel line of history in Git	14
Bug	A mistake in code that makes it behave wrongly	11
Cache	A tiny fast store between the CPU and RAM	1
Cell	One block of code or notes in a Jupyter notebook	6
Client / server	The side that asks / the side that answers	9
Cloud	Someone else's computers, rented over the network	12
Commit	A saved snapshot in Git, with a message saying why	14

TERM	PLAIN MEANING	CHAPTER
Compiler	Translates a whole program into machine code before running	6
Container	A miniature Linux system packaged with a program	8
CPU	The processor that carries out instructions	1
CSV	A table saved as plain text with comma-separated values	10
Database	A program that keeps related tables and answers questions about them	10
DataFrame	pandas' name for a table of data	15
Dependency	A package that another package needs	7
Dictionary	Values looked up by key; JSON's <code>{ }</code>	6
Directory	Another word for folder	3
DNS	Turns a domain name into an IP address	9
Documentation (docs)	The official explanation of how software or a library works	11
Driver	A small program the OS uses to talk to one device	2
Encoding (UTF-8)	The rule for turning bytes into characters	3
Environment	The interpreter, packages and versions your code runs with	8
Environment variable	A named setting the shell passes to programs, such as PATH	4
Function	A named, reusable block of code with inputs and outputs	6
Git	A tool that records the history of a project folder	14
Hash	A one-way scramble; how passwords are stored	12
IDE / editor	A program for writing code, such as VS Code	5, 6

TERM	PLAIN MEANING	CHAPTER
Import	Load an installed package into your program	7
Interpreter	Runs code line by line; Python is one	6
JSON	A plain-text format for nested data	9, 10
Jupyter notebook	A file of code and notes run cell by cell, with results shown inline	6
Kernel	The running Python process behind a notebook	6
Library / package	Reusable code written by others	7
List	Values kept in order, counted from 0	6
Localhost	Your own computer, as seen by itself on a network	9
Lock file	A complete, exact record of every installed package	8
Merge conflict	Two branches changed the same lines; you choose	14
Operating system	Software that manages hardware and runs other programs	2
PATH	The OS list of folders searched for commands	4, 5
Phishing	A message that imitates a trusted sender to steal a login	12
pip	Python's package manager	7
Pipe	Sends one command's output into another	4
Port	A numbered door on a server; 443 for https	9
Process	A program that is currently running	2
RAM	Fast, temporary working memory	1
Relative path	A file address starting from the current folder	3
Remote	A copy of a repository elsewhere, such as GitHub	14
Repository (repo)	A folder whose history Git is tracking	14

TERM	PLAIN MEANING	CHAPTER
Script	A .py file run from start to finish	6
SQL / SQLite	The language for asking databases questions / a database in one file	10
Standard library	Packages that come built into a language	7
Status code	The three-digit result of a web request; 200 is OK, 404 not found	9
Syntax	The spelling and grammar rules of a language	6
Terminal / shell	Typing commands to the OS	4
Thread	One line of work inside a process	2
Tidy data	One row per thing, one column per property, one value per cell	10
Traceback	Python's error report, read bottom-up	11
Two-factor (2FA)	A second proof of identity beyond the password	12
Type	What kind of value something is: number, text, list...	6
Variable	A named box holding a value	6
Virtual environment	A separate set of packages for one project	8
Working directory	The folder a program or terminal is currently "in"	3

Further reading and books to grow into

These go deeper than this book. The order follows the chapters above. Everything in the first list is free.

Courses and lessons, in order

1. **Alongside Chapters 1–2:** [Crash Course Computer Science](#), 40 short animated episodes from transistors to the internet.
2. **After Chapter 4:** [Software Carpentry: The Unix Shell](#), terminal lessons with real files.
3. **After Chapter 6:** [Harvard CS50P](#), a full free Python course with graded problem sets, and [Automate the Boring Stuff with Python](#), for everyday chores.
4. **After Chapter 8:** [The Missing Semester of Your CS Education](#) (MIT), the tools most courses skip.
5. **After Chapter 10:** [SQLBolt](#), interactive SQL in the browser, and the [pandas Getting started](#) tutorials.
6. **After Chapter 14:** [Pro Git](#), free, chapters 1–3, and GitHub's own [Hello World](#) guide.

Books to grow into

How computers work

- *Code: The Hidden Language of Computer Hardware and Software*, Charles Petzold (2nd ed., 2022). Builds a computer from switches upward. Its free companion site, [CodeHiddenLanguage.com](https://codehiddenlanguage.com), animates the circuits.
- *How Computers Really Work*, Matthew Justice. A hands-on tour of hardware and software, with small projects.
- *Operating Systems: Three Easy Pieces*, Remzi and Andrea Arpaci-Dusseau. Free. A clear, friendly OS textbook.
- *Computer Networking: A Top-Down Approach*, Kurose and Ross. The standard networking text; the authors' site has free lecture videos and slides.

Programming

- *Think Python*, Allen Downey (3rd ed.). Free. Defines every term as it appears; ideal after this book.
- *Automate the Boring Stuff with Python*, Al Sweigart. Free. Practical scripts for files, spreadsheets, PDFs and email.
- *The Pragmatic Programmer*, Hunt and Thomas. How to think about building software, once you are building some.

Data

- *Python for Data Analysis*, Wes McKinney (3rd ed.). Free online. Written by the creator of pandas.
- *Python Data Science Handbook*, Jake VanderPlas. Free. NumPy, pandas, matplotlib and machine-learning basics.
- *Storytelling with Data*, Cole Nussbaumer Knaflic. How to make a chart someone else understands.

Security

- *Have I Been Pwned* and the Electronic Frontier Foundation's Surveillance Self-Defense guides. Free, practical, kept up to date.

A last piece of advice: when a resource stops making sense, come back to the matching chapter here, re-read it, and then continue. The missing piece is usually context, and context is what this book is for.